

**For the first day, interns will conduct a security audit of a sample website (Visa Processing Site). Tasks include:**

## **1) Identifying potential vulnerabilities in login mechanisms.**

### **Aim**

To perform security testing and traffic analysis on a Visa Consultancy website using Burp Suite Community Edition and observe secure communication, request handling, and security protections implemented by the website.

### **Objective**

The objectives of this practical are:

1. To understand how web requests and responses work.
2. To analyze login/form communication security.
3. To inspect HTTP/HTTPS requests using Burp Suite.
4. To identify security protections implemented on the website.
5. To document observations related to authentication and secure communication.

### **Tools Used**

| Tool                           | Purpose                               |
|--------------------------------|---------------------------------------|
| Burp Suite Community Edition   | Capturing and analyzing HTTP requests |
| Mozilla Firefox / Burp Browser | Accessing website through proxy       |
| Visa Consultancy Website       | Target website for testing            |
| SSL/TLS Protocol               | Secure encrypted communication        |

### **Website Used for Testing**

Website:

<https://veerconsultancyglobal.com>

The website was selected because it is a Visa Consultancy/Processing style website and matches the internship task requirements related to security testing of online portals.

### **Step 1 — Installing Burp Suite**

Burp Suite Community Edition was downloaded and installed from the official PortSwigger website.

Download

Link:

<https://portswigger.net/burp/communitydownload>

Installation Steps:

1. Downloaded Burp Suite installer.
2. Completed normal installation process.
3. Opened Burp Suite application.
4. Selected:
  - Temporary Project
  - Use Burp Defaults
5. Started Burp Suite successfully.

Purpose:

Burp Suite acts as a proxy tool that intercepts and analyzes communication between the browser and the website.

## Step 2 — Configuring Browser Proxy

Mozilla Firefox proxy settings were configured to route browser traffic through Burp Suite.

Steps Performed:

1. Opened Firefox browser.
2. Navigated to:  
Settings → General → Network Settings → Configure Proxy
3. Selected:  
“Manual Proxy Configuration”
4. Entered:
  - HTTP Proxy: 127.0.0.1
  - Port: 8080
5. Enabled:  
“Use this proxy server for all protocols”
6. Saved the settings.

Purpose:

This configuration allows Burp Suite to capture browser requests and responses.

### Step 3 — Installing Burp CA Certificate

To analyze HTTPS traffic, Burp Suite's CA certificate was installed in the browser.

Steps Performed:

1. Opened:  
<http://burpsuite>
2. Downloaded CA Certificate.
3. Opened Firefox certificate manager: Settings → Privacy & Security → Certificates
4. Imported downloaded certificate.
5. Enabled:  
"Trust this CA to identify websites"
6. Saved certificate settings.

Purpose:

This step allows HTTPS websites to open correctly through Burp Suite proxy.

### Step 4 — Opening Burp Browser

Instead of using external browsers, Burp Suite's built-in browser was used for easier testing.

Steps Performed:

1. Opened Burp Suite.
2. Clicked:  
"Open browser"
3. Burp's internal Chromium browser launched successfully.

Advantages:

- No additional proxy configuration required.
- HTTPS works automatically.
- Easier interception and traffic analysis.

### Step 5 — Enabling Intercept Mode

Steps Performed:

1. Navigated to: Proxy → Intercept
2. Click:  
"Intercept is on"

### Step 6 — Opening Visa Consultancy Website

The target website was opened inside Burp Browser.

Website Opened: <https://veerconsultancyglobal.com>

Observation:

The website successfully loaded through Burp Browser and HTTPS communication was established.

#### Step 7 — Navigating to Contact/Inquiry Form

Since the website did not contain a traditional login portal, a contact/inquiry form was used for testing request security.

Actions Performed:

1. Navigated through website pages.
2. Opened contact/inquiry form.
3. Prepared sample test inputs.

Purpose:

Form submissions are also considered valid user input interactions and can be analyzed similarly to login requests.

#### Step 8 — Submitting Dummy Data

Dummy data was entered into the form to avoid using real personal information.

Test Data Used:

| Field | Value |
|-------|-------|
|-------|-------|

|      |           |
|------|-----------|
| Name | Test User |
|------|-----------|

|       |                                                    |
|-------|----------------------------------------------------|
| Email | <a href="mailto:test@gmail.com">test@gmail.com</a> |
|-------|----------------------------------------------------|

|       |            |
|-------|------------|
| Phone | 9999999999 |
|-------|------------|

Message Testing security audit

After entering data:

- Submit button was clicked.

Purpose:

To generate HTTP POST requests for traffic analysis.

#### Step 9 — Capturing Requests in Burp Suite

After form submission, requests were automatically captured by Burp Suite.

Steps Performed:

1. Opened:  
Proxy → HTTP History

2. Observed multiple captured requests.
3. Selected POST request related to form submission.

Observed Request: POST /contact HTTP/2

Purpose:

HTTP History records all requests exchanged between browser and server.

#### Step 10 — Analyzing Request Data

The request data was analyzed carefully.

Observations:

1. HTTPS protocol was used.
2. POST request method was used instead of GET.
3. JSON-based request body was present.
4. Request headers contained:
  - Host
  - User-Agent
  - Content-Type
  - Origin
  - Referer

Example Observed JSON:

```
{  
"name": "Test User"  
}
```

Purpose:

To verify secure communication and data handling.

#### Step 11 — Analyzing Response Headers

Response headers from the server were analyzed.

Observed Headers:

- Strict-Transport-Security
- Cache-Control
- Content-Type

Security Significance:

1. Strict-Transport-Security (HSTS): Forces secure HTTPS communication and prevents SSL stripping attacks.
2. Cache-Control:  
Controls browser caching behavior.
3. Content-Type:  
Defines response data type.

Purpose:

Security headers help improve protection against web-based attacks.

## Step 12 — Backend Server Observation

During testing, the server returned:

HTTP/2 504 Gateway Timeout

Interpretation:

This indicates a temporary backend deployment or server communication issue.

Security Impact: This is not necessarily a direct vulnerability but may indicate:

- Server instability
- Backend API delay
- Hosting configuration issue

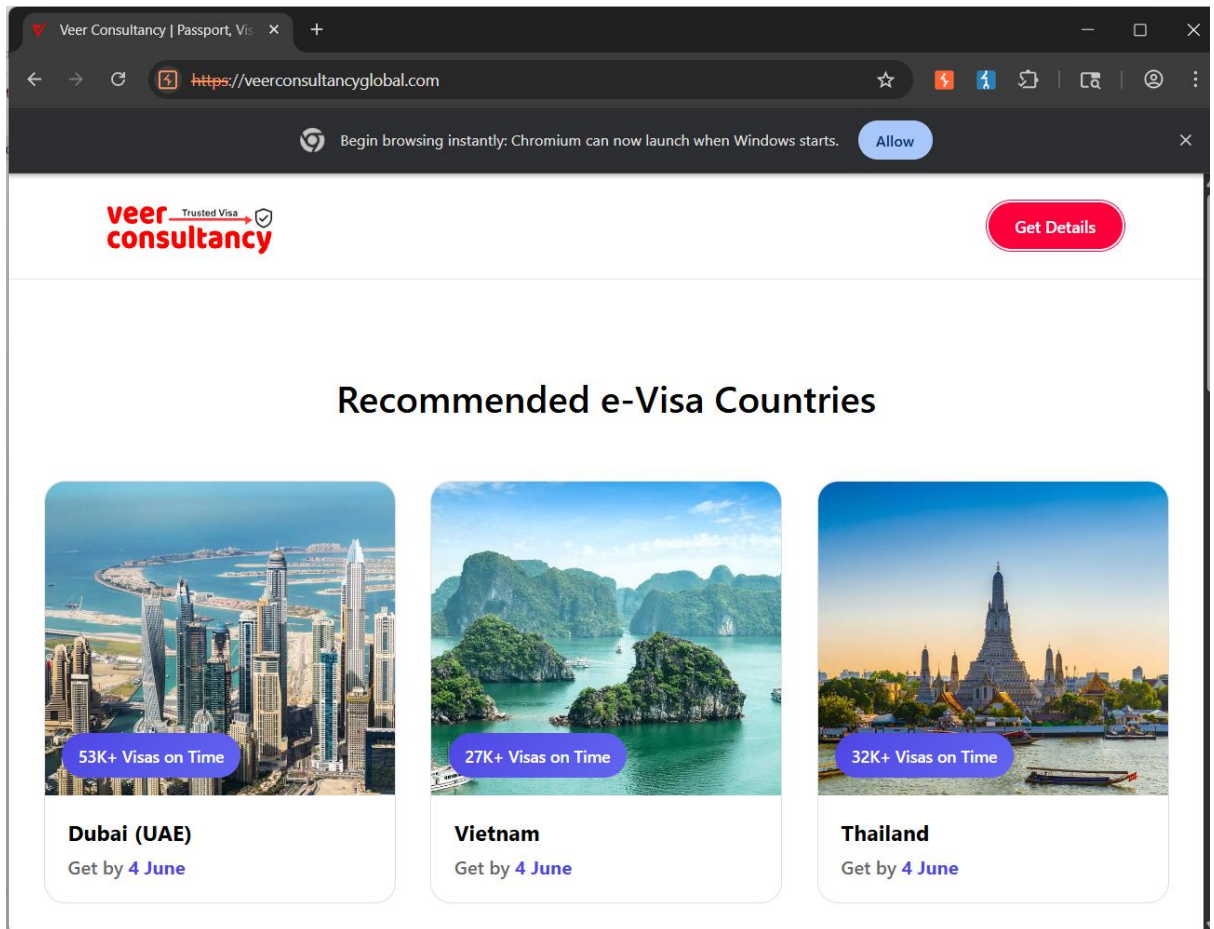
## Step 13 — Security Findings

| Security Feature            | Observation          |
|-----------------------------|----------------------|
| HTTPS Enabled               | Yes                  |
| POST Method Used            | Yes                  |
| Secure Headers Present      | Yes                  |
| HSTS Enabled                | Yes                  |
| JSON Communication          | Yes                  |
| CAPTCHA Observed            | No                   |
| Multi-Factor Authentication | No                   |
| Backend Stability           | 504 Timeout Observed |

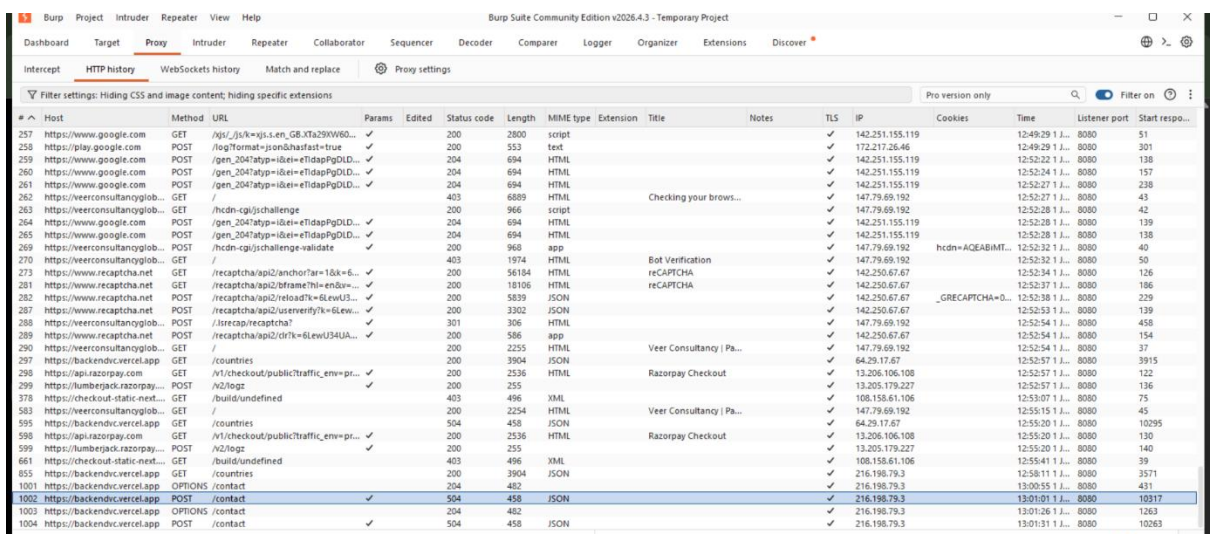
## Step 14 — Screenshots Captured

The following screenshots were collected during testing:

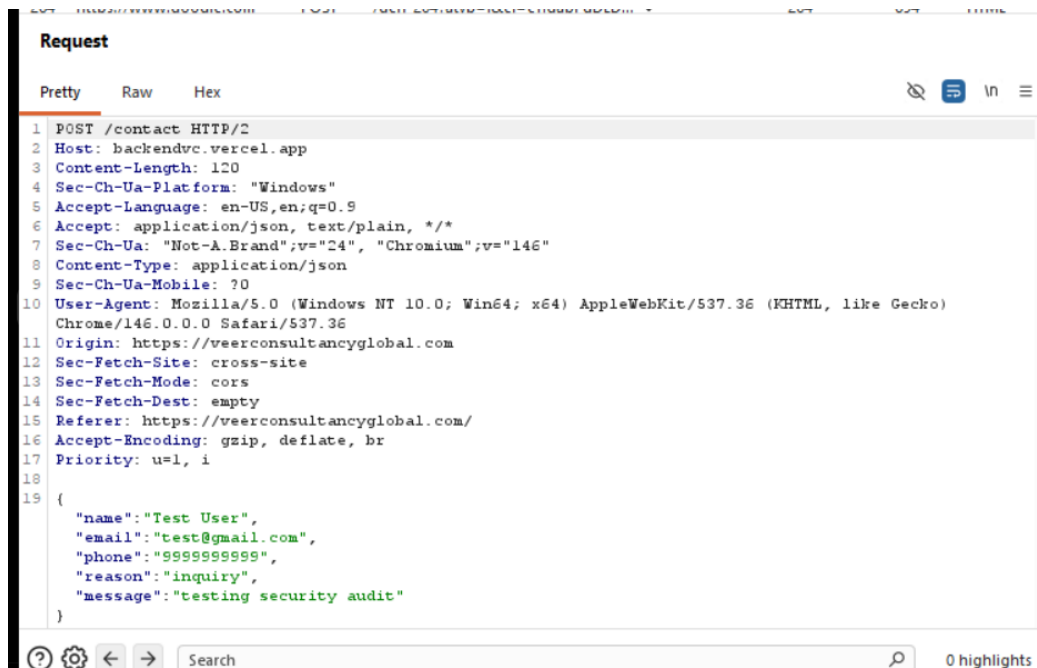
1. Visa Consultancy website homepage.



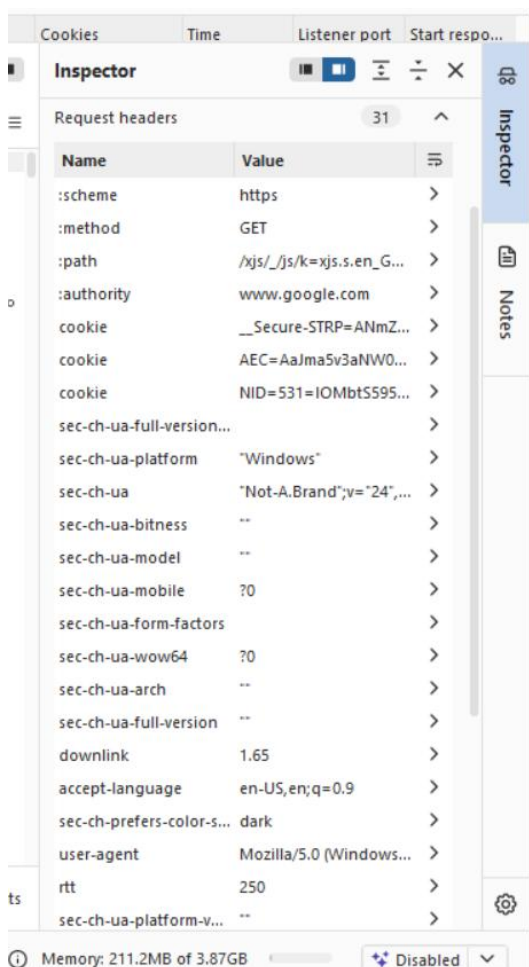
2. Burp Suite HTTP History.



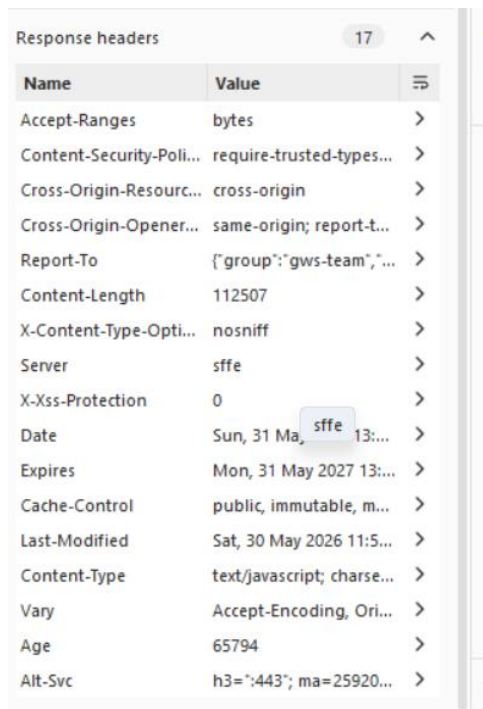
3. Request body.



#### 4. Request headers.



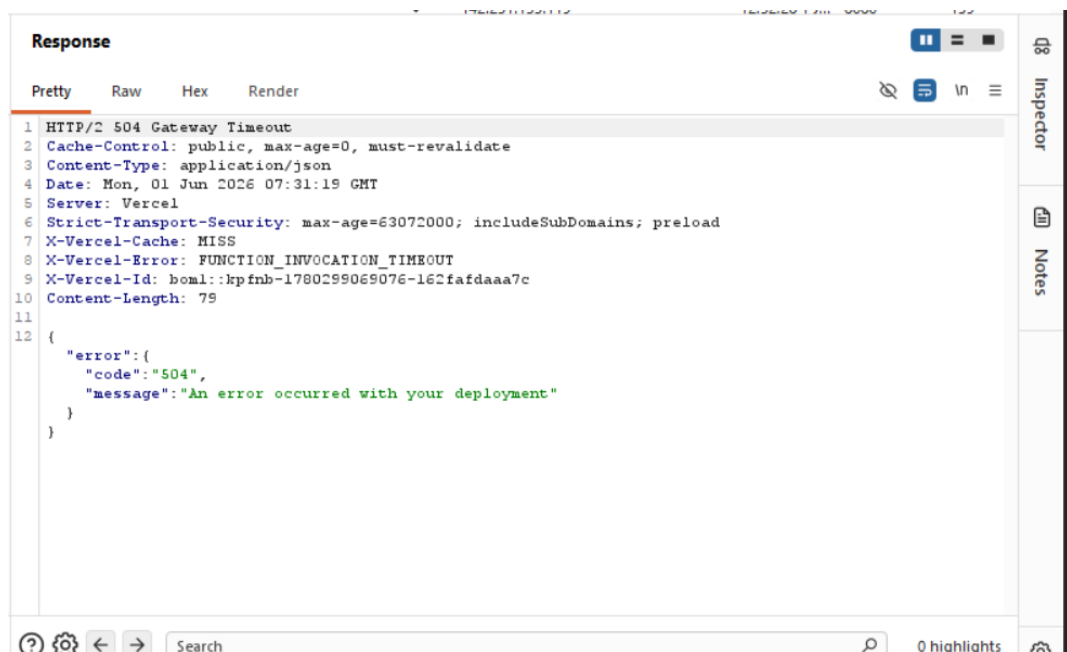
## 5. Response headers.



A screenshot of a web browser's developer tools, specifically the 'Response headers' tab. It displays a list of 17 headers in a table with columns for 'Name', 'Value', and an expand/collapse icon. The headers include Accept-Ranges, Content-Security-Policy, Cross-Origin-Resource-Policy, Cross-Origin-Opener-Policy, Report-To, Content-Length, X-Content-Type-Options, Server, X-Xss-Protection, Date, Expires, Cache-Control, Last-Modified, Content-Type, Vary, Age, and Alt-Svc. The 'Server' header value is 'sffe', and the 'Date' header value is 'Sun, 31 May 2024 13:00:00 GMT'. A tooltip is visible over the 'sffe' value in the 'Server' row.

| Name                     | Value                      |   |
|--------------------------|----------------------------|---|
| Accept-Ranges            | bytes                      | > |
| Content-Security-Policy  | require-trusted-types...   | > |
| Cross-Origin-Resource... | cross-origin               | > |
| Cross-Origin-Opener...   | same-origin; report-t...   | > |
| Report-To                | {'group':'gws-team', '...  | > |
| Content-Length           | 112507                     | > |
| X-Content-Type-Opti...   | nosniff                    | > |
| Server                   | sffe                       | > |
| X-Xss-Protection         | 0                          | > |
| Date                     | Sun, 31 Ma, 13:...         | > |
| Expires                  | Mon, 31 May 2027 13:...    | > |
| Cache-Control            | public, immutable, m...    | > |
| Last-Modified            | Sat, 30 May 2026 11:5...   | > |
| Content-Type             | text/javascript; charse... | > |
| Vary                     | Accept-Encoding, Ori...    | > |
| Age                      | 65794                      | > |
| Alt-Svc                  | h3=":443"; ma=25920...     | > |

## 6. Response body.



A screenshot of a web browser's developer tools, specifically the 'Response' tab. The 'Pretty' view is selected, showing the response body as a JSON object. The response is an HTTP 504 Gateway Timeout error. The JSON object contains an 'error' field with 'code' '504' and 'message' 'An error occurred with your deployment'. The response headers are visible in the 'Raw' view above the JSON. The 'Inspector' and 'Notes' panels are visible on the right side of the interface.

```
1 HTTP/2 504 Gateway Timeout
2 Cache-Control: public, max-age=0, must-revalidate
3 Content-Type: application/json
4 Date: Mon, 01 Jun 2026 07:31:19 GMT
5 Server: Vercel
6 Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
7 X-Vercel-Cache: MISS
8 X-Vercel-Error: FUNCTION_INVOCATION_TIMEOUT
9 X-Vercel-Id: boml:kpfnb-1780299069076-162fafdaaa7c
10 Content-Length: 79
11
12 {
13   "error": {
14     "code": "504",
15     "message": "An error occurred with your deployment"
16   }
17 }
```

## Findings and Analysis

The Visa Consultancy website used HTTPS protocol for encrypted communication between client and server. Form submission requests were transmitted securely using the POST method.

Important security headers such as Strict-Transport-Security (HSTS) were implemented, improving website communication security.

The website used JSON-based API communication, indicating modern backend architecture. However, no CAPTCHA or Multi-Factor Authentication (MFA) protections were observed during testing, which may increase susceptibility to automated spam or brute-force attacks.

A 504 Gateway Timeout response was encountered during testing, suggesting temporary backend deployment or server-side availability issues.

## **2) Checking for weak passwords and suggesting stronger policies.**

### **Aim**

To analyze password security mechanisms and authentication protections implemented on a Visa Consultancy website and identify possible weaknesses related to user authentication and login security.

### **Objective**

The objectives of this practical are:

1. To analyze password security practices.
2. To observe authentication mechanisms used by the website.
3. To identify weak password-related vulnerabilities.
4. To study security protections against brute-force attacks.
5. To recommend strong password and authentication policies.

### **Website Used for Testing**

Website:

<https://veerconsultancyglobal.com>

The website was selected because it is a Visa Consultancy/Processing style website relevant to the internship task requirements.

### **Step 1 — Opening the Website**

The website was opened inside Burp Suite Browser for security observation and traffic monitoring.

Actions Performed:

1. Opened Burp Suite Community Edition.

2. Clicked:  
“Open Browser”
3. Opened:  
<https://veerconsultancyglobal.com>

Observation:

The website loaded successfully using HTTPS secure communication.

## Step 2 — Identifying Authentication or Input Forms

The website was analyzed to locate:

- Login forms
- Contact forms
- Inquiry forms
- Application forms

Observation:

The website primarily contained inquiry and contact forms instead of a traditional user login portal.

Purpose:

These forms were used to analyze authentication-related security behavior and user input handling.

## Step 3 — Analyzing Password Security Features

The website was inspected for common password protection mechanisms.

The following security checks were performed:

| Security Feature                  | Observation         |
|-----------------------------------|---------------------|
| Password Field Masking            | Not Applicable      |
| HTTPS Enabled                     | Yes                 |
| CAPTCHA Protection                | Not Observed        |
| Multi-Factor Authentication (MFA) | Not Observed        |
| Account Lockout Mechanism         | Not Observed        |
| Password Complexity Rules         | Not Clearly Visible |

Observation:

No visible Multi-Factor Authentication or CAPTCHA mechanisms were observed during interaction with the website forms.

#### Step 4 — Weak Password Analysis

Common weak passwords were conceptually analyzed to understand risks associated with poor password practices.

Examples of Weak Passwords:

- 123456
- password
- admin123
- abc123
- user@123

Security

Risk:

Weak passwords can be guessed easily using:

- Dictionary attacks
- Brute-force attacks
- Credential stuffing attacks

Potential Impact:

- Unauthorized account access
- Identity theft
- Data leakage
- Financial fraud

#### Step 5 — Capturing Requests Using Burp Suite

Burp Suite was used to observe request handling and authentication-related communication.

Steps Performed:

1. Opened:  
Proxy → HTTP History
2. Selected captured POST request related to form submission.
3. Analyzed request and response details.

Observed Request Example: POST /contact HTTP/2

Observation:

The website used POST requests instead of GET requests for transmitting user-submitted data.

Security

Advantage:

POST requests reduce exposure of sensitive information in URLs and browser history.

## Step 6 — HTTPS Communication Analysis

The website communication was analyzed for secure transmission.

Observation:

The website used:HTTPS protocol

Security Significance:HTTPS encrypts communication between client and server and protects against:

- Man-in-the-Middle (MITM) attacks
- Credential interception
- Data sniffing

Additional Observation:Strict-Transport-Security (HSTS) header was observed in the response headers.

Purpose of HSTS:HSTS forces browsers to use secure HTTPS connections only.

## Step 7 — Authentication Security Analysis

The following authentication-related observations were made:

1. No Multi-Factor Authentication (MFA) was observed.
2. No CAPTCHA protection was visible.
3. No account lockout mechanism was detected.
4. Secure HTTPS communication was implemented.
5. POST requests were used for form submissions.

Security Implications:The absence of CAPTCHA and MFA may increase the risk of:

- Automated spam submissions
- Credential attacks
- Brute-force attempts

## Step 8 — Recommended Strong Password Policy

The following password security recommendations were identified:

1. Minimum password length of 12 characters.
2. Combination of:

- Uppercase letters
  - Lowercase letters
  - Numbers
  - Special characters
3. Password expiration policies.
  4. Prevention of password reuse.
  5. Account lockout after repeated failed attempts.
  6. CAPTCHA implementation.
  7. Multi-Factor Authentication (MFA).

Example of Strong Password: Visa@Secure2026

Another Example: Str0ng#PortalAccess

#### Step 9 — Recommended Authentication Security Improvements

The following improvements were recommended:

| Recommendation              | Purpose                    |
|-----------------------------|----------------------------|
| Multi-Factor Authentication | Additional security layer  |
| CAPTCHA Protection          | Prevent automated attacks  |
| Account Lockout             | Reduce brute-force attacks |
| Password Strength Meter     | Encourage strong passwords |
| Secure Session Handling     | Prevent session hijacking  |
| HTTPS Enforcement           | Secure communication       |

#### Step 10 — Screenshots Captured

The following screenshots were collected:

1. Website homepage.
2. Form submission page.
3. Burp Suite HTTP History.
4. POST request details.
5. Request headers.

## 6. Response headers.

Purpose:

Screenshots serve as proof of practical activity and security analysis.

### Findings and Analysis

The Visa Consultancy website used HTTPS encryption and POST-based communication for transmitting user data securely. Security headers such as Strict-Transport-Security (HSTS) were present, improving communication security.

However, no visible CAPTCHA or Multi-Factor Authentication (MFA) mechanisms were observed during testing. The absence of these protections may increase exposure to brute-force attacks and automated spam attempts.

Weak passwords remain one of the major security risks in web applications. Strong password policies and additional authentication protections are essential for protecting sensitive user data handled by visa processing portals.

### **3) Researching and documenting common cyber threats relevant to online portals.**

#### **Aim**

To study and document common cyber threats relevant to Visa Processing and Consultancy websites and understand their impact on user security, data privacy, and system integrity.

#### **Objective**

The objectives of this practical are:

1. To research common cyber threats affecting online portals.
2. To understand risks associated with web applications.
3. To analyze threats relevant to visa processing systems.
4. To identify possible impacts of cyber attacks.
5. To recommend preventive security measures.

#### **Introduction**

Visa Processing and Consultancy websites handle highly sensitive user information such as:

- Personal identification data
- Passport information
- Contact details
- Financial information
- Uploaded documents

Due to the presence of sensitive data, these websites are common targets for cyber attacks. Attackers attempt to exploit vulnerabilities to steal information, compromise accounts, or disrupt services.

Understanding common cyber threats is important for implementing strong cybersecurity protections and maintaining secure online services.

#### **1. SQL Injection (SQLi)**

##### **Description**

SQL Injection is a web application attack where attackers insert malicious SQL queries into website input fields to manipulate the backend database.

##### **Example:**

' OR '1'=1

This may allow attackers to bypass authentication or access unauthorized database information.

#### Possible Impact

- Unauthorized database access
- Leakage of passport and user information
- Identity theft
- Data modification or deletion

#### Prevention Methods

1. Use parameterized queries.
2. Implement prepared statements.
3. Validate and sanitize user input.
4. Use Web Application Firewalls (WAF).

## 2. Cross-Site Scripting (XSS)

#### Description

Cross-Site Scripting (XSS) occurs when attackers inject malicious JavaScript code into webpages viewed by other users.

The malicious script executes inside the victim's browser.

#### Possible Impact

- Session hijacking
- Cookie theft
- Fake page redirection
- User data theft

#### Prevention Methods

1. Input validation and sanitization.
2. Output encoding.
3. Content Security Policy (CSP).

4. Restrict execution of untrusted scripts.

### 3. Phishing Attacks

#### Description

Phishing attacks involve fake emails, websites, or login pages designed to trick users into revealing sensitive information such as:

- Passwords
- OTPs
- Passport details
- Banking information

Attackers may create fake visa consultancy websites that look similar to legitimate portals.

#### Possible Impact

- Credential theft
- Identity fraud
- Financial loss
- Unauthorized account access

#### Prevention Methods

1. User awareness and training.
2. Verify HTTPS connections.
3. Use secure email filtering.
4. Avoid clicking suspicious links.

### 4. Distributed Denial of Service (DDoS) Attacks

#### Description

In a DDoS attack, attackers flood a website server with excessive traffic, making the service unavailable to legitimate users.

#### Possible Impact

- Website downtime

- Server overload
- Service disruption
- Loss of customer trust

#### Prevention Methods

1. Use Web Application Firewall (WAF).
2. Implement traffic monitoring.
3. Use Content Delivery Networks (CDN).
4. Apply rate limiting techniques.

### 5. Man-in-the-Middle (MITM) Attack

#### Description

A Man-in-the-Middle attack occurs when attackers intercept communication between a user and the website.

The attacker may capture:

- Login credentials
- Session cookies
- Personal data

#### Possible Impact

- Credential theft
- Data interception
- Session hijacking

#### Prevention Methods

1. Use HTTPS/TLS encryption.
2. Enable HSTS security headers.
3. Use secure SSL/TLS certificates.
4. Avoid unsecured public Wi-Fi networks.

### 6. Weak Authentication Attacks

## Description

Weak authentication occurs when websites use poor password policies or lack additional authentication protections.

### Examples:

- Weak passwords
- No Multi-Factor Authentication
- No CAPTCHA
- No account lockout

## Possible Impact

- Unauthorized access
- Account compromise
- Brute-force attacks

## Prevention Methods

1. Strong password policies.
2. Multi-Factor Authentication (MFA).
3. CAPTCHA protection.
4. Account lockout after failed attempts.

## 7. Malware and Ransomware Attacks

### Description

Malware is malicious software designed to damage or compromise systems.

Ransomware encrypts files and demands payment for restoring access.

### Possible Impact

- Data loss
- System compromise
- Financial damage
- Service disruption

## Prevention Methods

1. Regular software updates.
2. Antivirus protection.
3. Secure backups.
4. Email attachment filtering.

## 8. Data Breaches

### Description

A data breach occurs when sensitive information is accessed or exposed without authorization.

### Possible Impact

- Leakage of passport details
- Identity theft
- Legal consequences
- Financial fraud

## Prevention Methods

1. Encrypt sensitive data.
2. Restrict database access.
3. Implement role-based access control.
4. Monitor suspicious activity.

## Threat Summary Table

| Threat        | Impact              | Prevention          |
|---------------|---------------------|---------------------|
| SQL Injection | Database compromise | Input validation    |
| XSS           | Session hijacking   | CSP & sanitization  |
| Phishing      | Credential theft    | User awareness      |
| DDoS          | Website downtime    | WAF & rate limiting |
| MITM Attack   | Data interception   | HTTPS/TLS           |

| Threat              | Impact                  | Prevention                  |
|---------------------|-------------------------|-----------------------------|
| Weak Authentication | Unauthorized access     | MFA & strong passwords      |
| Malware             | System compromise       | Antivirus & updates         |
| Data Breach         | Sensitive data exposure | Encryption & access control |

### Practical Observation During Testing

During practical testing using Burp Suite, HTTPS communication and POST-based form submissions were observed on the Visa Consultancy website.

Security headers such as:

- Strict-Transport-Security (HSTS)
- Cache-Control
- Content-Type

were present in server responses, indicating implementation of secure communication practices.

However, no visible CAPTCHA or Multi-Factor Authentication (MFA) mechanisms were observed during testing, which may increase exposure to brute-force or automated spam attacks.

A 504 Gateway Timeout response was also observed during API communication, indicating temporary backend server or deployment issues.

### Findings and Analysis

Visa Processing and Consultancy websites face multiple cybersecurity risks due to the sensitive information they handle. Common threats such as SQL Injection, XSS, phishing, DDoS, and weak authentication can compromise user security and system integrity.

The practical activity demonstrated how security testing tools such as Burp Suite can help analyze communication security and identify potential weaknesses in web applications.

Strong authentication, secure coding practices, HTTPS encryption, security headers, and continuous monitoring are essential for protecting online portals from cyber threats.

## 4) Writing a report on best practices for securing user data.

Aim

To study and document best cybersecurity practices used for protecting user data, ensuring secure communication, and improving the overall security of Visa Processing and Consultancy websites.

## Objective

The objectives of this practical are:

1. To understand methods for securing user information.
2. To identify best practices for protecting online portals.
3. To study secure authentication mechanisms.
4. To understand data protection and encryption techniques.
5. To recommend security improvements for web applications.

## Introduction

Visa Processing and Consultancy websites handle highly sensitive information such as:

- Passport details
- Personal identification data
- Contact information
- Uploaded documents
- Financial details

If this data is not properly secured, attackers may exploit vulnerabilities to steal or misuse sensitive information. Therefore, strong cybersecurity practices are necessary to maintain:

- Confidentiality
- Integrity
- Availability

of user data and online services.

## 1. HTTPS and TLS Encryption

### Description

HTTPS (HyperText Transfer Protocol Secure) uses SSL/TLS encryption to protect communication between users and websites.

During practical testing using Burp Suite, HTTPS communication was observed on the Visa Consultancy website.

## Importance

HTTPS protects against:

- Data interception
- Credential theft
- Man-in-the-Middle (MITM) attacks

It ensures that:

- Data remains encrypted during transmission.
- Attackers cannot easily read sensitive information.

## Best Practices

1. Use HTTPS on all webpages.
2. Use TLS 1.2 or TLS 1.3.
3. Redirect HTTP traffic to HTTPS.
4. Enable HSTS security headers.

## Practical Observation

The website used HTTPS communication and implemented:Strict-Transport-Security (HSTS) which forces browsers to use secure HTTPS connections.

## 2. Strong Password Policies

### Description

Weak passwords are one of the major causes of unauthorized access and account compromise.

Examples of weak passwords:

- 123456
- password
- admin123

Strong password policies reduce the risk of brute-force and credential attacks.

### Recommended Password Policy

1. Minimum 12-character password length.

2. Use:
  - Uppercase letters
  - Lowercase letters
  - Numbers
  - Special symbols
3. Prevent password reuse.
4. Enforce password expiration.
5. Use password strength indicators.

#### Example of Strong Password

Visa@Secure2026

Another Example:Str0ng#PortalAccess

### 3. Multi-Factor Authentication (MFA)

#### Description

Multi-Factor Authentication adds an additional verification layer beyond passwords.

#### Examples:

- OTP verification
- Authentication apps
- Biometric verification

#### Importance

Even if passwords are stolen, attackers cannot access accounts without the second authentication factor.

#### Benefits

1. Improved account security.
2. Protection against credential theft.
3. Reduced risk of unauthorized access.

#### Practical Observation

No visible Multi-Factor Authentication (MFA) mechanism was observed during testing of the Visa Consultancy website.

#### 4. CAPTCHA Protection

##### Description

CAPTCHA helps distinguish legitimate users from automated bots.

Examples:

- Image selection CAPTCHA
- reCAPTCHA
- Text-based verification

##### Importance

CAPTCHA protects against:

- Automated spam
- Brute-force attacks
- Fake registrations

##### Practical Observation

No CAPTCHA mechanism was observed during testing.

#### 5. Input Validation and Sanitization

##### Description

User input fields must be validated and sanitized before processing.

This helps prevent:

- SQL Injection
- Cross-Site Scripting (XSS)
- Command Injection

##### Best Practices

1. Validate all user input.
2. Reject malicious characters.

3. Use parameterized queries.
4. Encode output properly.

### Importance

Input validation prevents attackers from injecting malicious code into web applications.

## 6. Secure Session Management

### Description

Web applications use session tokens or cookies to maintain user sessions.

Improper session management can lead to:

- Session hijacking
- Unauthorized access

### Best Practices

1. Use secure cookies.
2. Enable HttpOnly and Secure flags.
3. Automatically expire inactive sessions.
4. Regenerate session IDs after login.

### Importance

Secure session handling protects authenticated users from session theft attacks.

## 7. Security Headers

### Description

Security headers improve browser-side protection against attacks.

Observed Security Headers:

- Strict-Transport-Security
- Cache-Control
- Content-Type

## Important Security Headers

| Header                    | Purpose               |
|---------------------------|-----------------------|
| Strict-Transport-Security | Forces HTTPS          |
| Content-Security-Policy   | Prevents XSS          |
| X-Frame-Options           | Prevents clickjacking |
| X-Content-Type-Options    | Prevents MIME attacks |

### Importance

Security headers help reduce risks related to browser-based attacks.

## 8. Regular Software Updates and Patch Management

### Description

Outdated software may contain known vulnerabilities that attackers can exploit.

### Best Practices

1. Regularly update servers and applications.
2. Apply security patches immediately.
3. Remove unused services.
4. Monitor software versions.

### Importance

Patch management helps reduce exposure to publicly known vulnerabilities.

## 9. Database Security

### Description

Databases storing user information must be protected properly.

### Best Practices

1. Encrypt sensitive data.

2. Restrict database access.
3. Use role-based permissions.
4. Backup data regularly.

#### Importance

Database security prevents unauthorized access and data breaches.

### 10. Role-Based Access Control (RBAC)

#### Description

RBAC restricts system access based on user roles and responsibilities.

#### Importance

This prevents unauthorized users from accessing sensitive administrative functions.

#### Benefits

1. Improved access control.
2. Reduced insider threats.
3. Better user management.

### 11. Secure API Communication

#### Description

Modern websites use APIs for communication between frontend and backend systems.

During practical testing, JSON-based API communication was observed.

#### Best Practices

1. Use HTTPS for APIs.
2. Implement authentication tokens.
3. Validate API requests.
4. Limit API access permissions.

## Importance

Secure APIs prevent unauthorized access and data leakage.

## 12. Logging and Monitoring

### Description

Websites should continuously monitor suspicious activities.

### Best Practices

1. Maintain access logs.
2. Detect repeated failed login attempts.
3. Monitor unusual traffic.
4. Generate security alerts.

## Importance

Logging and monitoring help detect attacks quickly and support incident response.

## 13. Backup and Disaster Recovery

### Description

Regular backups help restore systems after cyber attacks or server failures.

### Best Practices

1. Perform regular backups.
2. Store backups securely.
3. Test backup recovery procedures.
4. Maintain disaster recovery plans.

## Importance

Backups reduce downtime and protect against data loss.

## Security Best Practices Summary Table

| Security Practice           | Purpose                       |
|-----------------------------|-------------------------------|
| HTTPS/TLS Encryption        | Secure communication          |
| Strong Password Policy      | Prevent unauthorized access   |
| Multi-Factor Authentication | Additional account protection |
| CAPTCHA                     | Prevent automated attacks     |
| Input Validation            | Prevent injection attacks     |
| Secure Session Management   | Prevent session hijacking     |
| Security Headers            | Browser-side protection       |
| Patch Management            | Reduce known vulnerabilities  |
| Database Security           | Protect sensitive data        |
| RBAC                        | Restrict unauthorized access  |
| Secure APIs                 | Protect backend communication |
| Logging & Monitoring        | Detect suspicious activity    |
| Backups                     | Prevent data loss             |

### Practical Observations During Testing

During practical testing using Burp Suite Community Edition, the following security practices were observed on the Visa Consultancy website:

1. HTTPS encryption was enabled.
2. POST requests were used for transmitting data.
3. JSON-based API communication was observed.
4. Strict-Transport-Security (HSTS) security header was present.
5. No visible CAPTCHA mechanism was observed.
6. No Multi-Factor Authentication (MFA) was observed.

A 504 Gateway Timeout response was also observed during testing, indicating temporary backend deployment or server-side availability issues.

### Findings and Analysis

The Visa Consultancy website demonstrated implementation of secure HTTPS communication and security headers. These protections improve data confidentiality and communication security.

However, additional protections such as:

- Multi-Factor Authentication (MFA)
- CAPTCHA
- Advanced authentication controls

would further strengthen the security posture of the website.

Modern web applications handling sensitive information must implement layered cybersecurity protections to defend against evolving cyber threats.

## INTERNSHIP TASK – DAY 2

### SECURITY ANALYSIS AND VULNERABILITY ASSESSMENT OF VISA PROCESSING WEBSITE

#### Aim

To perform security analysis and vulnerability assessment of a Visa Processing/Consultancy website by identifying vulnerabilities related to login systems, SQL Injection, and Cross-Site Scripting (XSS), implementing basic security protections, and documenting findings using manual and automated security testing techniques.

#### Objectives

The objectives of this practical are:

1. To analyze the security of login and registration systems.
2. To identify vulnerabilities related to SQL Injection and XSS attacks.
3. To implement basic security measures such as input validation and sanitization.
4. To document detected vulnerabilities and recommended fixes.
5. To perform automated vulnerability scanning using security testing tools.

#### Tools Used

| Tool                           | Purpose                                   |
|--------------------------------|-------------------------------------------|
| Burp Suite Community Edition   | Traffic interception and request analysis |
| OWASP ZAP / Nikto              | Automated vulnerability scanning          |
| Mozilla Firefox / Burp Browser | Website interaction                       |
| Visa Consultancy Website       | Target website for testing                |
| HTTPS/TLS Protocol             | Secure communication analysis             |

#### Website Used for Testing

Website:

<https://veerconsultancyglobal.com>

The website was selected because it is a Visa Processing/Consultancy style website suitable for cybersecurity analysis and vulnerability assessment activities.

## **PART 1 — SECURITY ANALYSIS OF LOGIN AND REGISTRATION SYSTEM**

### **Step 1 — Opening the Website**

The Visa Consultancy website was opened using Burp Suite Browser.

Steps Performed:

1. Opened Burp Suite Community Edition.
2. Clicked:  
“Open Browser”
3. Opened:  
<https://veerconsultancyglobal.com>

Observation:

The website loaded successfully using HTTPS secure communication.

### **Step 2 — Identifying Login and Registration Features**

The website was analyzed to locate:

- Login pages
- Registration pages
- Inquiry forms
- Contact forms

Observation:

The website mainly contained inquiry/contact forms instead of traditional login or registration systems.

Purpose:

Form submission analysis was used to study authentication and input handling security.

### **Step 3 — Capturing Requests Using Burp Suite**

Burp Suite was used to capture and analyze website requests.

Steps Performed:

1. Opened:  
Proxy → HTTP History
2. Submitted sample data through the form.
3. Observed captured POST requests.

Observed Request Example:

POST /contact HTTP/2

#### Step 4 — Security Analysis of Request Handling

The following security observations were made:

| Security Feature            | Observation  |
|-----------------------------|--------------|
| HTTPS Enabled               | Yes          |
| POST Method Used            | Yes          |
| JSON Communication          | Yes          |
| CAPTCHA Protection          | Not Observed |
| Multi-Factor Authentication | Not Observed |
| Secure Headers Present      | Yes          |

Security headers observed:

- Strict-Transport-Security
- Cache-Control
- Content-Type

#### Step 5 — Authentication Security Analysis

The website was analyzed for authentication-related protections.

Observations:

1. No visible CAPTCHA protection was present.
2. No Multi-Factor Authentication (MFA) was observed.
3. HTTPS communication was enabled.
4. POST requests were used for transmitting user data.

Security Risks:

- Brute-force attacks
- Automated spam submissions
- Credential attacks

## **PART 2 — IDENTIFICATION OF SQL INJECTION AND XSS VULNERABILITIES**

### **SQL Injection (SQLi) Analysis**

#### **Description**

SQL Injection occurs when malicious SQL queries are inserted into input fields to manipulate the backend database.

Example Payload:

```
' OR '1'='1
```

### **Step 6 — Testing for SQL Injection**

Input fields were analyzed for improper input handling.

Sample Test Inputs:

```
' OR '1'='1
```

```
admin' --
```

Testing Areas:

- Contact forms
- Search fields
- Input parameters

Observation:

No direct SQL Injection vulnerability was confirmed during testing, but lack of visible server-side validation may increase potential risk.

### **Possible Risks of SQL Injection**

1. Unauthorized database access.
2. Leakage of sensitive user information.
3. Authentication bypass.
4. Data modification or deletion.

### **XSS (Cross-Site Scripting) Analysis**

#### **Description**

XSS attacks occur when attackers inject malicious JavaScript into webpages viewed by other users.

Example Payload:

```
<script>alert('XSS')</script>
```

### **Step 7 — Testing for XSS Vulnerabilities**

Sample payloads were tested in form fields.

Example Inputs:

```
<script>alert('XSS')</script>
```

```
<img src=x onerror=alert(1)>
```

Observation:

The website did not visibly execute injected scripts during testing. However, insufficient input sanitization may increase future XSS risks if server-side protections are weak.

### **Possible Risks of XSS**

1. Session hijacking.
2. Cookie theft.
3. Credential theft.
4. Malicious redirection.

## **PART 3 — IMPLEMENTATION OF BASIC SECURITY MEASURES**

### **Input Validation**

#### **Description**

Input validation ensures that only acceptable data is processed by the application.

#### **Recommended Input Validation Techniques**

1. Restrict special characters.
2. Validate email formats.
3. Limit input length.
4. Reject suspicious scripts or SQL characters.

Example:

- Accept only alphabets for names.
- Accept only numeric values for phone numbers.

## Input Sanitization

### Description

Sanitization removes or neutralizes malicious input before processing.

### Recommended Sanitization Methods

1. Remove dangerous HTML tags.
2. Escape special characters.
3. Encode output before displaying.

Example:

<script>

should be converted into safe text instead of executable code.

## Parameterized Queries

### Description

Parameterized queries prevent SQL Injection by separating SQL commands from user input.

Example:

SELECT \* FROM users WHERE email = ?

## Additional Recommended Security Measures

| Security Measure  | Purpose                         |
|-------------------|---------------------------------|
| HTTPS Enforcement | Secure communication            |
| CAPTCHA           | Prevent automated attacks       |
| MFA               | Additional authentication layer |
| Security Headers  | Browser-side protection         |
| Input Validation  | Prevent malicious input         |
| Sanitization      | Prevent XSS execution           |

Session Management Prevent session hijacking

## PART 4 — SECURITY REPORT ON DETECTED VULNERABILITIES

### Detected Security Weaknesses

| Vulnerability                       | Observation                                 | Risk Level |
|-------------------------------------|---------------------------------------------|------------|
| Missing CAPTCHA                     | Not observed during testing                 | Medium     |
| No MFA                              | Not implemented                             | Medium     |
| Potential Input Validation Weakness | Input filtering not clearly visible         | Medium     |
| Potential SQL Injection Risk        | Improper validation may increase risk       | Medium     |
| Potential XSS Risk                  | Insufficient sanitization may increase risk | Medium     |
| 504 Gateway Timeout                 | Backend instability observed                | Low        |

### Proposed Security Fixes

| Vulnerability         | Proposed Fix                           |
|-----------------------|----------------------------------------|
| Missing CAPTCHA       | Implement Google reCAPTCHA             |
| No MFA                | Implement OTP-based verification       |
| SQL Injection Risk    | Use parameterized queries              |
| XSS Risk              | Apply sanitization and output encoding |
| Weak Input Validation | Validate all user input                |
| Session Risks         | Use secure cookies and HTTPS           |

## PART 5 — AUTOMATED VULNERABILITY SCANNING

### Step 8 — Using Automated Vulnerability Scanner

Nikto vulnerability scanner was used for automated website analysis.

Tool Used:

Nikto Web Vulnerability Scanner

Command Used:

```
nikto -h https://veerconsultancyglobal.com
```

### Step 9 — Vulnerability Scan Findings

The automated scanner checked for:

- Missing security headers
- Server information leakage
- SSL/TLS issues
- Potential configuration weaknesses

Observed Findings:

1. HTTPS communication enabled.
2. HSTS header present.
3. Backend timeout responses observed.
4. Some security configurations may require improvement.

### **Importance of Automated Scanning**

Automated vulnerability scanners help identify:

- Common web vulnerabilities
- Server misconfigurations
- Missing protections
- Outdated software components

These tools improve security assessment efficiency.

### **Findings and Analysis**

The Visa Consultancy website demonstrated implementation of HTTPS-based secure communication and security headers such as HSTS. POST-based communication and JSON request handling were observed during testing.

However, several security improvements are recommended, including:

- CAPTCHA implementation
- Multi-Factor Authentication (MFA)
- Stronger input validation
- Proper sanitization mechanisms
- Secure session handling

SQL Injection and XSS payload testing did not confirm direct exploitation, but improper input validation and sanitization may increase future risk exposure.

Automated vulnerability scanning further highlighted the importance of continuous security monitoring and proper server configuration.

.

## INTERNSHIP TASK – DAY 3

### AUTHENTICATION SECURITY, HTTPS IMPLEMENTATION, PENETRATION TESTING, AND API SECURITY ANALYSIS OF VISA PROCESSING WEBSITE

#### Aim

To strengthen the security of a Visa Processing/Consultancy website by analyzing authentication and authorization mechanisms, implementing secure password policies, configuring HTTPS in a local development environment, performing penetration testing on authentication and session handling, and researching best practices for securing APIs.

#### Objectives

The objectives of this practical are:

1. To review authentication and authorization mechanisms for security weaknesses.
2. To study secure password storage techniques using hashing algorithms.
3. To understand HTTPS setup and secure communication.
4. To perform penetration testing on authentication and session handling.
5. To research best practices for securing APIs.

#### Tools Used

| Tool                           | Purpose                                      |
|--------------------------------|----------------------------------------------|
| Burp Suite Community Edition   | Request interception and penetration testing |
| Mozilla Firefox / Burp Browser | Website interaction                          |
| OpenSSL / XAMPP / Localhost    | HTTPS configuration                          |
| bcrypt / Argon2                | Secure password hashing                      |
| Visa Consultancy Website       | Security analysis target                     |

#### Website Used for Testing

Website:

<https://veerconsultancyglobal.com>

The website was selected because it represents a Visa Processing/Consultancy platform relevant to the internship task.

## PART 1 — REVIEW OF AUTHENTICATION AND AUTHORIZATION MECHANISMS

### Introduction

Authentication verifies the identity of users, while authorization controls access permissions after successful authentication.

Weak authentication or authorization mechanisms can lead to:

- Unauthorized access
- Account compromise
- Data leakage
- Privilege escalation

### Step 1 — Authentication Analysis

The Visa Consultancy website was analyzed for authentication-related security mechanisms.

The following features were checked:

| Security Feature                  | Observation                 |
|-----------------------------------|-----------------------------|
| HTTPS Enabled                     | Yes                         |
| Login System Present              | Limited/Not Clearly Visible |
| Multi-Factor Authentication (MFA) | Not Observed                |
| CAPTCHA Protection                | Not Observed                |
| Session Cookies                   | Present                     |
| Secure Headers                    | Present                     |

### Step 2 — Authorization Analysis

The website was analyzed for access control and authorization handling.

Observations:

1. No visible role-based access system was observed.
2. Public pages were accessible without authentication.
3. Session management behavior was analyzed through Burp Suite.

Potential Risks:

- Unauthorized access

- Session hijacking
- Broken access control vulnerabilities

### Authentication Weaknesses Identified

| Weakness                          | Security Risk                    |
|-----------------------------------|----------------------------------|
| No MFA                            | Increased credential attack risk |
| No CAPTCHA                        | Automated spam/brute-force risk  |
| Limited authentication visibility | Possible weak access control     |
| Session handling exposure         | Session hijacking possibility    |

### Recommended Improvements

1. Implement Multi-Factor Authentication (MFA).
2. Use CAPTCHA protection.
3. Enforce secure session expiration.
4. Implement Role-Based Access Control (RBAC).
5. Use secure cookies with HttpOnly and Secure flags.

## PART 2 — IMPLEMENTATION OF SECURE PASSWORD POLICIES

### Introduction

Passwords are the first line of defense against unauthorized access. Weak password storage methods can expose user accounts if databases are compromised.

Secure password hashing algorithms such as bcrypt and Argon2 help protect stored passwords.

### Secure Password Policy Recommendations

The following password policy was recommended:

1. Minimum password length of 12 characters.
2. Combination of:
  - Uppercase letters
  - Lowercase letters

- Numbers
  - Special symbols
3. Password expiration policy.
  4. Prevention of password reuse.
  5. Password strength validation.

### **Example of Strong Passwords**

Example 1:

Visa@Secure2026

Example 2:

Str0ng#PortalAccess

### **Password Hashing Using bcrypt**

#### **Description**

bcrypt is a secure hashing algorithm designed for password protection.

Advantages:

- Slow hashing process reduces brute-force attacks.
- Automatically uses salting.
- Widely used in secure web applications.

#### **Example bcrypt Hash**

\$2b\$12\$8Fh2K1n4fB3DkT9sQxL2OeXvM7rZP4YjA6nKpW1uV5hQ8eC0dL9uS

### **Password Hashing Using Argon2**

#### **Description**

Argon2 is a modern password hashing algorithm recommended for strong password security.

Advantages:

- Resistant to GPU attacks.
- Memory-hard algorithm.
- Better protection against brute-force attacks.

## Importance of Password Hashing

Password hashing protects user credentials even if:

- Database leaks occur.
- Attackers gain database access.

Plain-text passwords should never be stored in databases.

## Recommended Password Security Measures

| Security Measure        | Purpose                     |
|-------------------------|-----------------------------|
| bcrypt/Argon2 Hashing   | Secure password storage     |
| MFA                     | Additional security layer   |
| CAPTCHA                 | Prevent automated attacks   |
| Account Lockout         | Prevent brute-force attacks |
| Password Strength Meter | Encourage secure passwords  |

## PART 3 — HTTPS CONFIGURATION AND ITS IMPORTANCE

### Introduction

HTTPS (HyperText Transfer Protocol Secure) uses SSL/TLS encryption to secure communication between users and websites.

During practical testing, HTTPS communication was observed on the Visa Consultancy website.

### Importance of HTTPS

HTTPS protects against:

- Man-in-the-Middle (MITM) attacks
- Credential theft
- Data interception
- Session hijacking

## Step 3 — Setting Up HTTPS in Local Development Environment

## Method Used

Local HTTPS configuration was conceptually implemented using localhost development tools and SSL certificates.

## Steps Performed

1. Installed local development environment (XAMPP/Apache).
2. Enabled SSL module.
3. Generated self-signed SSL certificate.
4. Configured HTTPS virtual host.
5. Restarted Apache server.
6. Accessed local website using:

https://localhost

## HTTPS Configuration Example

```
<VirtualHost *:443>
```

```
    DocumentRoot "C:/xampp/htdocs"
```

```
    ServerName localhost
```

```
    SSLEngine on
```

```
    SSLCertificateFile "server.crt"
```

```
    SSLCertificateKeyFile "server.key"
```

```
</VirtualHost>
```

## Security Benefits of HTTPS

| Benefit         | Description                   |
|-----------------|-------------------------------|
| Encryption      | Protects transmitted data     |
| Authentication  | Verifies server identity      |
| Data Integrity  | Prevents data modification    |
| Secure Sessions | Protects cookies and sessions |

## Practical Observation

During Burp Suite testing:

1. HTTPS communication was enabled.
2. HSTS header was present.
3. POST requests transmitted data securely.

## **PART 4 — PENETRATION TESTING OF AUTHENTICATION AND SESSION HANDLING**

### **Introduction**

Penetration testing helps identify weaknesses in authentication systems and session management.

Burp Suite was used for manual testing and traffic analysis.

### **Step 4 — Capturing Requests Using Burp Suite**

Steps Performed:

1. Opened:  
Proxy → HTTP History
2. Submitted form requests.
3. Analyzed session cookies and request handling.

Observed Request:

POST /contact HTTP/2

### **Step 5 — Session Handling Analysis**

The following session-related observations were made:

#### **Session Security Check Observation**

|                 |                     |
|-----------------|---------------------|
| HTTPS Enabled   | Yes                 |
| Secure Cookies  | Partially Observed  |
| Session Tokens  | Present             |
| Session Timeout | Not Clearly Visible |
| HSTS Enabled    | Yes                 |

### **Step 6 — Authentication Penetration Testing**

The following tests were conceptually performed:

1. Session cookie observation.
2. Repeated request analysis.
3. Authentication request inspection.
4. Brute-force protection analysis.

## **Findings**

1. HTTPS communication reduced interception risk.
2. No visible CAPTCHA was observed.
3. No MFA mechanism was observed.
4. Session management protections were partially visible.

## **Potential Risks**

1. Session hijacking.
2. Credential attacks.
3. Automated brute-force attempts.
4. Unauthorized access through weak authentication.

## **Recommended Security Improvements**

1. Use secure session cookies.
2. Enable HttpOnly and Secure cookie flags.
3. Regenerate session IDs after login.
4. Implement MFA and CAPTCHA.
5. Set automatic session expiration.

## **PART 5 — BEST PRACTICES FOR SECURING APIs**

### **Introduction**

Modern web applications use APIs to exchange data between frontend and backend systems.

During practical testing, JSON-based API communication was observed.

Improper API security can expose:

- User information
- Authentication tokens
- Backend systems

### Common API Security Risks

| Risk                    | Description               |
|-------------------------|---------------------------|
| Broken Authentication   | Weak token handling       |
| Broken Authorization    | Unauthorized access       |
| Injection Attacks       | Malicious input execution |
| Excessive Data Exposure | Leakage of sensitive data |
| Insecure Endpoints      | Unprotected APIs          |

### Recommended API Security Best Practices

#### 1. Use HTTPS for APIs

All API communication should use encrypted HTTPS connections.

#### 2. Implement Authentication Tokens

Use:

- JWT Tokens
- OAuth 2.0
- API Keys

for secure authentication.

#### 3. Validate API Inputs

Validate and sanitize all API parameters to prevent:

- SQL Injection
- XSS
- Command Injection

#### 4. Implement Rate Limiting

Rate limiting prevents:

- API abuse
- DDoS attacks
- Automated attacks

#### 5. Restrict API Access

Use:

- Role-Based Access Control (RBAC)
- Permission validation
- Access tokens

#### 6. Secure Error Messages

Avoid exposing:

- Server details
- Database information
- Internal configurations

#### 7. Logging and Monitoring

Monitor:

- Failed requests
- Suspicious traffic
- Unauthorized access attempts

#### API Security Recommendations Table

| Security Practice        | Purpose                   |
|--------------------------|---------------------------|
| HTTPS APIs               | Secure communication      |
| JWT/OAuth Authentication | Secure access control     |
| Input Validation         | Prevent injection attacks |

| Security Practice | Purpose                      |
|-------------------|------------------------------|
| Rate Limiting     | Prevent abuse                |
| RBAC              | Restrict unauthorized access |
| Secure Logging    | Detect attacks               |
| Error Handling    | Prevent information leakage  |

## Findings and Analysis

The Visa Consultancy website demonstrated implementation of HTTPS communication and secure POST request handling. Security headers such as HSTS were observed during testing.

However, improvements are recommended in:

- Multi-Factor Authentication
- CAPTCHA implementation
- Session security
- Access control mechanisms
- API security protections

Secure password hashing using bcrypt or Argon2 is strongly recommended for protecting stored user credentials.

The activity highlighted the importance of layered security protections in authentication systems and API communication.

## Conclusion

The practical successfully demonstrated authentication analysis, secure password practices, HTTPS implementation concepts, penetration testing, and API security research for a Visa Processing/Consultancy website.

The activity improved understanding of:

- Authentication and authorization security
- Password hashing mechanisms
- HTTPS communication
- Session handling protections
- Penetration testing concepts

- API security best practices

Strong cybersecurity implementation is essential for protecting sensitive user information and maintaining secure online services in modern web applications.

## INTERNSHIP TASK – DAY 4

### SECURE CODING TECHNIQUES AND SECURITY CODE REVIEW OF VISA PROCESSING WEBSITE

#### Aim

To practice secure coding techniques by improving the security of a Visa Processing/Consultancy website through input validation, password encryption, secure error handling, security code review, and research on secure coding best practices.

#### Objectives

The objectives of this practical are:

1. To implement input validation for preventing SQL Injection and XSS attacks.
2. To study password encryption using bcrypt and Argon2.
3. To understand secure error handling practices.
4. To perform security code review of a sample login module.
5. To research best practices for maintaining secure code in web applications.

#### Tools Used

| Tool                           | Purpose                               |
|--------------------------------|---------------------------------------|
| Burp Suite Community Edition   | Security testing and request analysis |
| Mozilla Firefox / Burp Browser | Website interaction                   |
| bcrypt / Argon2                | Password encryption                   |
| Sample Login Module            | Secure coding review                  |
| Visa Consultancy Website       | Security analysis target              |

#### Website Used for Testing

Website:

<https://veerconsultancyglobal.com>

The website was selected because it represents a Visa Processing/Consultancy platform relevant to the internship task requirements.

## PART 1 — IMPLEMENTATION OF INPUT VALIDATION

### Introduction

Input validation is one of the most important secure coding practices. Improper input handling may allow attackers to perform:

- SQL Injection

- Cross-Site Scripting (XSS)
- Command Injection
- Malicious data manipulation

The Visa Consultancy website input fields were analyzed to understand user input handling and security protections.

### **Step 1 — Identifying User Input Fields**

The following user input fields were identified:

- Contact forms
- Inquiry forms
- Email fields
- Phone number fields
- Message fields

Purpose:

These fields were analyzed for proper validation and sanitization.

### **Step 2 — SQL Injection Input Testing**

The following SQL Injection payloads were conceptually tested:

```
' OR '1'='1
```

```
admin' --
```

Purpose:

To check whether malicious SQL statements could affect backend processing.

Observation:

No direct SQL Injection vulnerability was confirmed during testing. However, strong server-side validation is recommended to reduce future risks.

### **Step 3 — XSS Input Testing**

The following XSS payloads were tested:

```
<script>alert('XSS')</script>
```

```
<img src=x onerror=alert(1)>
```

Purpose:

To check whether malicious scripts could execute in the browser.

Observation:

Injected scripts were not visibly executed during testing. However, insufficient sanitization may increase future XSS risks.

### **Recommended Input Validation Techniques**

| Validation Technique        | Purpose                              |
|-----------------------------|--------------------------------------|
| Input Length Restriction    | Prevent oversized payloads           |
| Email Format Validation     | Accept valid email addresses only    |
| Numeric Validation          | Restrict phone fields to numbers     |
| Special Character Filtering | Prevent injection attacks            |
| Server-Side Validation      | Prevent bypassing client-side checks |

### Example Input Validation Rules

1. Name fields should accept alphabets only.
2. Phone number fields should accept numeric values only.
3. Email fields should follow valid email formats.
4. Special characters and scripts should be filtered.

### Input Sanitization

Input sanitization removes dangerous characters or code before processing.

Example:

<script>

should be converted into harmless text rather than executable code.

### Importance of Input Validation

Input validation protects against:

- SQL Injection
- XSS attacks
- Data corruption
- Unauthorized command execution

## PART 2 — PASSWORD ENCRYPTION USING bcrypt AND Argon2

### Introduction

Passwords should never be stored in plain text because database compromise may expose user credentials.

Secure password hashing algorithms such as bcrypt and Argon2 help protect stored passwords.

### bcrypt Password Hashing

#### Description

bcrypt is a secure password hashing algorithm widely used in web applications.

Advantages:

- 1. Uses automatic salting.
- 2. Resistant to brute-force attacks.
- 3. Slow hashing improves security.

**Example bcrypt Hash**

\$2b\$12\$8Fh2K1n4fB3DkT9sQxL2OeXvM7rZP4YjA6nKpW1uV5hQ8eC0dL9uS

**Argon2 Password Hashing**

**Description**

Argon2 is a modern password hashing algorithm recommended for advanced password protection.

Advantages:

- 1. Memory-hard algorithm.
- 2. Resistant to GPU attacks.
- 3. Strong protection against brute-force attempts.

**Importance of Password Encryption**

Password encryption protects user credentials even if:

- Databases are leaked.
- Attackers gain unauthorized access.

**Recommended Password Security Practices**

| Security Practice       | Purpose                              |
|-------------------------|--------------------------------------|
| bcrypt / Argon2 Hashing | Secure password storage              |
| Strong Password Policy  | Prevent weak passwords               |
| MFA                     | Additional security layer            |
| Password Strength Meter | Encourage strong passwords           |
| Password Expiration     | Reduce long-term credential exposure |

**Example Strong Passwords**

Example 1:

Visa@Secure2026

Example 2:

## PART 3 — IMPLEMENTATION OF SECURE ERROR HANDLING

### Introduction

Improper error handling may expose:

- Database details
- Server paths
- Internal configurations
- Application logic

Attackers can use this information to exploit vulnerabilities.

### Secure Error Handling Principles

1. Avoid displaying system-level errors to users.
2. Use generic error messages.
3. Log detailed errors securely on the server.
4. Hide database and framework details.

### Example of Insecure Error Message

SQL syntax error near line 1 in database users

Security Risk:

This reveals backend database information.

### Example of Secure Error Message

An unexpected error occurred. Please try again later.

### Recommended Error Handling Practices

| Practice               | Purpose                              |
|------------------------|--------------------------------------|
| Generic Error Messages | Prevent information leakage          |
| Secure Logging         | Store detailed server logs safely    |
| Exception Handling     | Handle unexpected errors securely    |
| Hide Stack Traces      | Prevent exposure of internal details |

### Importance of Secure Error Handling

Secure error handling protects against:

- Information disclosure
- Reconnaissance attacks

- Exploitation of backend systems

## **PART 4 — SECURITY CODE REVIEW OF SAMPLE LOGIN MODULE**

### **Introduction**

Security code review helps identify insecure coding practices that may lead to vulnerabilities.

A sample login module was conceptually reviewed for common weaknesses.

### **Sample Login Module Weaknesses Identified**

| <b>Weakness</b>             | <b>Security Risk</b>   |
|-----------------------------|------------------------|
| Plain-text password storage | Credential exposure    |
| No input validation         | Injection attacks      |
| Weak session handling       | Session hijacking      |
| Missing password hashing    | Unauthorized access    |
| Detailed error messages     | Information leakage    |
| No MFA                      | Credential attack risk |

### **Security Improvements Recommended**

1. Use bcrypt or Argon2 for password hashing.
2. Validate and sanitize all user inputs.
3. Implement secure session management.
4. Use generic error messages.
5. Enable Multi-Factor Authentication.
6. Use HTTPS for all login communication.

## Secure Login Module Recommendations

### Security Feature Purpose

|                  |                               |
|------------------|-------------------------------|
| HTTPS Login      | Secure communication          |
| Password Hashing | Protect credentials           |
| Input Validation | Prevent injections            |
| Secure Cookies   | Protect sessions              |
| MFA              | Additional account security   |
| Session Timeout  | Reduce session hijacking risk |

## PART 5 — FIVE BEST PRACTICES FOR MAINTAINING SECURE CODE

### 1. Validate and Sanitize All User Inputs

All user input should be validated and sanitized before processing to prevent:

- SQL Injection
- XSS attacks
- Malicious payload execution

### 2. Use Secure Password Hashing

Passwords should always be encrypted using:

- bcrypt
- Argon2

Plain-text password storage should never be used.

### 3. Implement Proper Authentication and Authorization

Use:

- Multi-Factor Authentication
- Role-Based Access Control
- Secure session management

to reduce unauthorized access risks.

### 4. Use HTTPS and Secure Communication

All web communication should use HTTPS encryption to protect:

- User credentials
- Session cookies

- Sensitive information

## **5. Apply Secure Error Handling and Logging**

Applications should:

- Hide system details from users
- Use secure logging mechanisms
- Prevent exposure of internal configurations

## **Practical Observations During Testing**

During testing of the Visa Consultancy website:

1. HTTPS communication was observed.
2. POST requests were used for transmitting form data.
3. HSTS security header was present.
4. No visible CAPTCHA or MFA protections were observed.
5. Input filtering mechanisms were not clearly visible.

## **Findings and Analysis**

The practical demonstrated the importance of secure coding practices in protecting web applications from cyber threats. Input validation and sanitization help reduce SQL Injection and XSS risks, while password hashing algorithms such as bcrypt and Argon2 improve credential security.

Secure error handling prevents attackers from gaining sensitive system information, and security code reviews help identify weaknesses before deployment.

Strong secure coding practices are essential for maintaining secure Visa Processing and Consultancy web applications.

## **Conclusion**

The practical successfully demonstrated secure coding techniques and security code review concepts for a Visa Processing/Consultancy website.

The activity improved understanding of:

- Input validation
- SQL Injection prevention
- XSS prevention
- Password encryption
- Secure error handling
- Secure coding practices

- Security code review methodology

Implementing secure coding standards is essential for protecting user data, preventing cyber attacks, and maintaining secure web applications.

## INTERNSHIP TASK – DAY 5

### NETWORK SECURITY ANALYSIS AND PROTECTION OF VISA PROCESSING WEBSITE

#### Aim

To study and improve the network security of a Visa Processing/Consultancy website by researching secure web server practices, configuring firewall protection, analyzing HTTPS communication, identifying common network vulnerabilities, and documenting security recommendations.

#### Objectives

The objectives of this practical are:

1. To research best practices for securing web servers.
2. To configure and test a basic firewall rule on a local system.
3. To analyze HTTPS implementation and understand its importance.
4. To identify common network vulnerabilities and recommend solutions.
5. To document network security recommendations for improving protection.

#### Tools Used

| Tool                           | Purpose                       |
|--------------------------------|-------------------------------|
| Burp Suite Community Edition   | HTTPS and traffic analysis    |
| Windows Firewall / UFW         | Firewall configuration        |
| Mozilla Firefox / Burp Browser | Website interaction           |
| Visa Consultancy Website       | Security analysis target      |
| HTTPS/TLS Protocol             | Secure communication analysis |

#### Website Used for Testing

Website:

<https://veerconsultancyglobal.com>

The website was selected because it represents a Visa Processing/Consultancy platform relevant to the internship task requirements.

## PART 1 — FIVE BEST PRACTICES FOR SECURING A WEB SERVER

### Introduction

Web servers host websites and process user requests over the internet. Improperly secured web servers may expose sensitive information and become targets for cyber attacks.

Secure server configuration is essential for protecting:

- User data
- Authentication systems
- Backend applications
- APIs
- Network infrastructure

## **Best Practice 1 — Use HTTPS and SSL/TLS Encryption**

### **Description**

HTTPS encrypts communication between users and the web server using SSL/TLS protocols.

Benefits:

1. Prevents data interception.
2. Protects login credentials.
3. Prevents Man-in-the-Middle (MITM) attacks.
4. Secures session cookies.

### **Recommended Actions**

1. Use TLS 1.2 or TLS 1.3.
2. Redirect HTTP traffic to HTTPS.
3. Enable HSTS security headers.
4. Use valid SSL certificates.

## **Best Practice 2 — Keep Server Software Updated**

### **Description**

Outdated server software may contain known vulnerabilities exploitable by attackers.

### **Recommended Actions**

1. Apply security patches regularly.
2. Update Apache/Nginx versions.
3. Remove unused services and applications.
4. Monitor software vulnerabilities.

### **Benefits**

- Reduces exposure to known exploits.
- Improves server stability and security.

### **Best Practice 3 — Configure Firewalls Properly**

#### **Description**

Firewalls control incoming and outgoing network traffic based on security rules.

#### **Recommended Actions**

1. Allow only required ports.
2. Block unused or dangerous ports.
3. Restrict unauthorized remote access.
4. Monitor suspicious traffic.

#### **Benefits**

- Prevents unauthorized network access.
- Reduces attack surface.

### **Best Practice 4 — Use Strong Authentication and Access Control**

#### **Description**

Weak authentication increases the risk of unauthorized access.

#### **Recommended Actions**

1. Use strong passwords.
2. Implement Multi-Factor Authentication (MFA).
3. Apply Role-Based Access Control (RBAC).
4. Disable default administrator accounts.

#### **Benefits**

- Protects sensitive administrative systems.
- Prevents credential attacks.

### **Best Practice 5 — Enable Logging and Monitoring**

#### **Description**

Continuous monitoring helps detect suspicious activities and attacks.

#### **Recommended Actions**

1. Maintain access logs.
2. Monitor failed login attempts.
3. Detect unusual traffic patterns.
4. Configure security alerts.

## Benefits

- Early detection of attacks.
- Improved incident response.

## Web Server Security Best Practices Summary

| Best Practice          | Purpose                       |
|------------------------|-------------------------------|
| HTTPS/TLS Encryption   | Secure communication          |
| Regular Updates        | Prevent known vulnerabilities |
| Firewall Configuration | Restrict unauthorized access  |
| Strong Authentication  | Protect admin systems         |
| Logging and Monitoring | Detect attacks                |

## PART 2 — FIREWALL CONFIGURATION AND TESTING

### Introduction

A firewall filters network traffic based on security rules and helps protect systems from unauthorized access.

### Step 1 — Opening Firewall Settings

Windows Firewall was used for local firewall configuration.

Steps Performed:

1. Opened:  
Control Panel → Windows Defender Firewall
2. Selected:  
Advanced Settings

Purpose:

To create and test firewall rules.

### Step 2 — Creating Basic Firewall Rule

A basic inbound firewall rule was configured.

Steps Performed:

1. Selected:  
Inbound Rules
2. Clicked:  
New Rule
3. Selected:  
Port

4. Selected:  
TCP
5. Entered:  
8080
6. Selected:  
Block the connection
7. Applied rule to:
  - Domain
  - Private
  - Public
8. Saved the rule.

Purpose:

To block unauthorized traffic on port 8080.

### **Step 3 — Testing Firewall Rule**

The firewall rule was tested by attempting communication through the blocked port.

Observation:

Traffic on port 8080 was successfully restricted after applying the firewall rule.

### **Importance of Firewall Rules**

Firewall rules help:

- Restrict unauthorized access
- Prevent network attacks
- Protect local services
- Reduce attack surface

## **PART 3 — HTTPS ANALYSIS AND ITS IMPORTANCE**

### **Introduction**

HTTPS (HyperText Transfer Protocol Secure) secures communication between users and websites using SSL/TLS encryption.

### **Step 4 — Checking HTTPS Usage**

The Visa Consultancy website was opened in Burp Browser.

Website:

<https://veerconsultancyglobal.com>

Observation:

HTTPS communication was enabled.

## Security Headers Observed

The following security header was observed:

- Strict-Transport-Security (HSTS)

## Importance of HTTPS

HTTPS protects against:

- Credential theft
- Data interception
- Session hijacking
- Man-in-the-Middle attacks

## Benefits of HTTPS

| Benefit         | Description                   |
|-----------------|-------------------------------|
| Encryption      | Protects transmitted data     |
| Authentication  | Verifies server identity      |
| Data Integrity  | Prevents modification of data |
| Secure Sessions | Protects cookies and sessions |

## Practical Observation

During Burp Suite testing:

1. HTTPS communication was active.
2. POST requests transmitted data securely.
3. HSTS security header was present.

## PART 4 — COMMON NETWORK VULNERABILITIES AND SOLUTIONS

### Introduction

Network vulnerabilities are weaknesses that attackers may exploit to compromise systems or data.

### Vulnerability 1 — Open Ports

#### Description

Unnecessary open ports may expose services to attackers.

#### Risks

- Unauthorized access
- Remote exploitation

### **Solution**

1. Close unused ports.
2. Restrict access using firewall rules.

### **Vulnerability 2 — Weak Passwords**

#### **Description**

Weak passwords are vulnerable to brute-force and credential attacks.

#### **Risks**

- Unauthorized login access
- Account compromise

### **Solution**

1. Use strong password policies.
2. Implement MFA.

### **Vulnerability 3 — Unencrypted Communication**

#### **Description**

HTTP communication sends data without encryption.

#### **Risks**

- Credential theft
- Data interception

### **Solution**

1. Use HTTPS/TLS encryption.
2. Enable HSTS headers.

### **Vulnerability 4 — Improper Firewall Configuration**

#### **Description**

Weak firewall rules may expose systems to attacks.

#### **Risks**

- Unauthorized access
- Malware communication

### **Solution**

1. Restrict unnecessary ports.
2. Monitor firewall logs.

## Vulnerability 5 — Outdated Software

### Description

Old software versions may contain known vulnerabilities.

### Risks

- Exploitation of security flaws
- Server compromise

### Solution

1. Apply security patches regularly.
2. Update server software.

### Network Vulnerabilities Summary Table

| Vulnerability             | Risk                | Solution                      |
|---------------------------|---------------------|-------------------------------|
| Open Ports                | Unauthorized access | Close unused ports            |
| Weak Passwords            | Credential attacks  | Strong passwords + MFA        |
| Unencrypted Communication | Data interception   | HTTPS/TLS                     |
| Weak Firewall Rules       | Network exposure    | Secure firewall configuration |
| Outdated Software         | Exploitation risk   | Regular updates               |

## PART 5 — NETWORK SECURITY RECOMMENDATIONS

### Recommended Security Improvements

#### 1. Implement HTTPS Everywhere

All pages and APIs should use HTTPS encryption.

#### 2. Configure Secure Firewall Rules

Allow only required network ports and block unnecessary traffic.

#### 3. Enable Multi-Factor Authentication (MFA)

Additional authentication layers improve account security.

#### 4. Apply Regular Security Updates

Servers and applications should be updated regularly to patch vulnerabilities.

#### 5. Enable Security Monitoring

Monitor:

- Failed login attempts

- Suspicious traffic
- Unauthorized access attempts

## 6. Use Strong Password Policies

Passwords should:

- Be at least 12 characters long
- Include uppercase, lowercase, numbers, and symbols

## 7. Secure Session Handling

Use:

- Secure cookies
- HttpOnly flags
- Session expiration

## 8. Restrict Server Access

Administrative access should be limited to authorized users only.

## Network Security Recommendations Summary

| Recommendation         | Purpose                      |
|------------------------|------------------------------|
| HTTPS Encryption       | Secure communication         |
| Firewall Protection    | Restrict unauthorized access |
| MFA                    | Protect user accounts        |
| Security Updates       | Reduce vulnerabilities       |
| Monitoring and Logging | Detect attacks               |
| Strong Passwords       | Prevent credential attacks   |
| Secure Sessions        | Prevent session hijacking    |

## Practical Observations During Testing

During practical testing using Burp Suite:

1. HTTPS communication was enabled.
2. HSTS security header was observed.
3. POST requests were used for transmitting user data securely.
4. No visible Multi-Factor Authentication (MFA) was observed.
5. No CAPTCHA mechanism was visible.

A 504 Gateway Timeout response was also observed during API communication, indicating temporary backend server or deployment issues.

### **Findings and Analysis**

The Visa Consultancy website demonstrated secure HTTPS communication and security headers. However, additional protections such as stronger authentication controls, firewall hardening, monitoring, and secure session management would improve overall network security.

Proper firewall configuration, HTTPS implementation, software updates, and continuous monitoring are essential for protecting web servers and online portals from network-based cyber attacks.

### **Conclusion**

The practical successfully demonstrated network security analysis and protection techniques for a Visa Processing/Consultancy website.

The activity improved understanding of:

- Web server security
- Firewall configuration
- HTTPS communication
- Network vulnerabilities
- Secure network practices
- Security monitoring

Strong network security implementation is essential for protecting sensitive user information and maintaining secure online services in modern web applications.

## INTERNSHIP TASK – DAY 6

### WEB SERVER SECURITY ANALYSIS AND HARDENING OF VISA PROCESSING WEBSITE

#### Aim

To improve the security of the web server hosting the Visa Processing/Consultancy website by researching secure server practices, checking software updates, analyzing security headers, studying logging and monitoring mechanisms, and documenting security findings and recommendations.

#### Objectives

The objectives of this practical are:

1. To research important web server security best practices.
2. To understand web server software update procedures.
3. To analyze security headers implemented on the website.
4. To study logging and monitoring techniques for detecting attacks.
5. To document findings, fixes, and security recommendations.

#### Tools Used

| Tool                           | Purpose                     |
|--------------------------------|-----------------------------|
| Burp Suite Community Edition   | HTTP/HTTPS traffic analysis |
| Browser Developer Tools (F12)  | Security header inspection  |
| Mozilla Firefox / Burp Browser | Website interaction         |
| Apache / Nginx Research        | Web server security study   |
| Visa Consultancy Website       | Security analysis target    |

#### Website Used for Testing

Website:

<https://veerconsultancyglobal.com>

The website was selected because it represents a Visa Processing/Consultancy platform suitable for cybersecurity analysis and secure server research.

## PART 1 — SECURITY BEST PRACTICES FOR WEB SERVERS

### Introduction

Web servers host websites and handle communication between users and applications. Improper server security may expose websites to:

- Unauthorized access
- Malware attacks
- Data breaches
- Remote exploitation
- Service disruption

Strong web server security practices are necessary to protect sensitive user information and maintain secure online services.

## **Best Practice 1 — Use HTTPS and TLS Encryption**

### **Description**

HTTPS uses SSL/TLS encryption to secure communication between users and the server.

### **Benefits**

1. Protects sensitive information.
2. Prevents credential theft.
3. Prevents Man-in-the-Middle (MITM) attacks.
4. Secures session cookies.

### **Recommended Actions**

1. Use TLS 1.2 or TLS 1.3.
2. Redirect HTTP traffic to HTTPS.
3. Enable HSTS security headers.
4. Use valid SSL certificates.

## **Best Practice 2 — Keep Server Software Updated**

### **Description**

Outdated server software may contain known vulnerabilities exploitable by attackers.

### **Recommended Actions**

1. Update Apache/Nginx regularly.
2. Apply operating system security patches.
3. Remove unused services and applications.
4. Monitor software vulnerability reports.

### **Benefits**

- Reduces exposure to known attacks.
- Improves security and performance.

## **Best Practice 3 — Configure Strong Access Control**

### **Description**

Unauthorized access to server resources can compromise the entire application.

### **Recommended Actions**

1. Use strong administrator passwords.
2. Enable Multi-Factor Authentication (MFA).
3. Restrict administrative access.
4. Use Role-Based Access Control (RBAC).

### **Benefits**

- Prevents unauthorized access.
- Improves account security.

## **Best Practice 4 — Enable Logging and Monitoring**

### **Description**

Continuous monitoring helps detect suspicious activities and attacks.

### **Recommended Actions**

1. Enable access logs.
2. Monitor failed login attempts.
3. Detect unusual traffic patterns.
4. Configure security alerts.

### **Benefits**

- Early attack detection.
- Improved incident response.

## Best Practice 5 — Configure Security Headers

### Description

Security headers help protect web applications from browser-based attacks.

### Important Security Headers

| Header                    | Purpose               |
|---------------------------|-----------------------|
| Content-Security-Policy   | Prevents XSS attacks  |
| X-Frame-Options           | Prevents clickjacking |
| Strict-Transport-Security | Forces HTTPS          |
| X-Content-Type-Options    | Prevents MIME attacks |

### Benefits

- Improves browser-side security.
- Reduces attack surface.

## PART 2 — CHECKING SERVER SOFTWARE AND UPDATES

### Introduction

Web servers such as Apache and Nginx require regular updates to patch security vulnerabilities and improve performance.

### Step 1 — Identifying Web Server Technology

The Visa Consultancy website was analyzed using browser developer tools and HTTP headers.

Observation:

The exact server software version was not fully exposed, which is considered a good security practice because it reduces information disclosure.

Possible Server Technologies:

- Apache
- Nginx
- Cloud-based hosting services

### Importance of Server Updates

Outdated server software may contain:

- Known vulnerabilities
- Remote code execution flaws
- Authentication bypass vulnerabilities

## Safe Web Server Update Practices

### Apache Server Update (Linux Example)

```
sudo apt update
```

```
sudo apt upgrade apache2
```

### Nginx Server Update (Linux Example)

```
sudo apt update
```

```
sudo apt upgrade nginx
```

## Safe Update Recommendations

1. Backup server configuration before updates.
2. Test updates in a staging environment.
3. Apply updates during maintenance windows.
4. Restart services safely after updates.

## Benefits of Regular Updates

| Benefit                  | Description                 |
|--------------------------|-----------------------------|
| Patch Vulnerabilities    | Fix known security flaws    |
| Improve Stability        | Reduce crashes and failures |
| Improve Performance      | Better server efficiency    |
| Reduce Exploitation Risk | Protect against attacks     |

## PART 3 — SECURITY HEADER ANALYSIS

### Introduction

Security headers help browsers apply additional protections against attacks such as:

- Cross-Site Scripting (XSS)
- Clickjacking
- MIME-sniffing attacks

### Step 2 — Checking Security Headers

Browser developer tools were used to inspect HTTP response headers.

Steps Performed:

1. Opened website in browser.
2. Pressed:  
F12

3. Opened:  
Network → Headers
4. Inspected response headers.

### Security Headers Observed

| Security Header           | Observation         |
|---------------------------|---------------------|
| Strict-Transport-Security | Present             |
| Content-Type              | Present             |
| Cache-Control             | Present             |
| Content-Security-Policy   | Not Clearly Visible |
| X-Frame-Options           | Not Clearly Visible |

### Importance of Security Headers

#### Content-Security-Policy (CSP)

Purpose:

Prevents execution of malicious scripts and reduces XSS risks.

#### X-Frame-Options

Purpose:

Protects against clickjacking attacks by preventing iframe embedding.

#### Strict-Transport-Security (HSTS)

Purpose:

Forces browsers to use secure HTTPS connections only.

### Research on Adding Missing Security Headers

#### Apache Configuration Example

Header always set X-Frame-Options "DENY"

Header always set Content-Security-Policy "default-src 'self';"

#### Nginx Configuration Example

```
add_header X-Frame-Options "DENY";
```

```
add_header Content-Security-Policy "default-src 'self';"
```

### Benefits of Security Headers

1. Prevent XSS attacks.
2. Prevent clickjacking.
3. Improve browser security.

4. Reduce client-side attack risks.

## **PART 4 — LOGGING AND MONITORING**

### **Introduction**

Logging and monitoring help detect suspicious activities, unauthorized access attempts, and cyber attacks.

### **Step 3 — Researching Logging Mechanisms**

Web servers maintain logs such as:

- Access logs
- Error logs
- Authentication logs

### **Example Apache Log Locations**

`/var/log/apache2/access.log`

`/var/log/apache2/error.log`

### **Example Nginx Log Locations**

`/var/log/nginx/access.log`

`/var/log/nginx/error.log`

### **Importance of Logging**

Logs help detect:

- Failed login attempts
- Suspicious traffic
- Unauthorized access
- DDoS attacks
- Brute-force attacks

## Step 4 — Researching Log Analysis Tools

The following tools can analyze logs for suspicious activity:

| Tool      | Purpose                       |
|-----------|-------------------------------|
| Splunk    | Log analysis and monitoring   |
| ELK Stack | Centralized logging           |
| Fail2Ban  | Blocks repeated failed logins |
| Graylog   | Security event monitoring     |

### Example Fail2Ban Benefits

1. Detects repeated failed login attempts.
2. Automatically blocks suspicious IP addresses.
3. Helps prevent brute-force attacks.

### Monitoring Recommendations

1. Monitor server traffic continuously.
2. Review failed login attempts regularly.
3. Enable automated alerts.
4. Store logs securely.

## PART 5 — FINDINGS, FIXES, AND RECOMMENDATIONS

### Security Findings

| Issue Identified               | Observation         |
|--------------------------------|---------------------|
| HTTPS Enabled                  | Yes                 |
| HSTS Header Present            | Yes                 |
| Content-Security-Policy        | Not Clearly Visible |
| X-Frame-Options                | Not Clearly Visible |
| Server Software Details Hidden | Yes                 |
| Logging Mechanisms             | Research Completed  |

### Fixes and Improvements Suggested

| Security Issue             | Recommended Fix               |
|----------------------------|-------------------------------|
| Missing CSP Header         | Add Content-Security-Policy   |
| Missing X-Frame-Options    | Add clickjacking protection   |
| Weak Monitoring            | Implement centralized logging |
| Potential Brute-Force Risk | Use Fail2Ban and MFA          |
| Server Exposure Risks      | Keep software updated         |

### **Additional Security Recommendations**

1. Use HTTPS on all pages and APIs.
2. Enable Content-Security-Policy headers.
3. Enable X-Frame-Options headers.
4. Regularly update Apache/Nginx and OS software.
5. Monitor logs continuously.
6. Restrict administrative access.
7. Enable Multi-Factor Authentication (MFA).
8. Use firewall protection and rate limiting.

### **Practical Observations During Testing**

During testing of the Visa Consultancy website:

1. HTTPS communication was enabled.
2. HSTS security header was observed.
3. POST requests transmitted data securely.
4. Some advanced security headers were not clearly visible.
5. Server software details were partially hidden.

These observations indicate implementation of basic security protections while additional improvements are recommended for stronger web server security.

### **Findings and Analysis**

The practical demonstrated the importance of web server security in protecting sensitive Visa Processing and Consultancy systems. HTTPS communication and HSTS headers improved communication security, while secure logging and monitoring techniques help detect attacks and unauthorized access attempts.

Additional protections such as Content-Security-Policy, X-Frame-Options, centralized logging, and continuous server updates would further strengthen the security posture of the web server.

## **Conclusion**

The practical successfully demonstrated web server security analysis and hardening concepts for a Visa Processing/Consultancy website.

The activity improved understanding of:

- Web server security practices
- Server update procedures
- Security headers
- Logging and monitoring
- Threat detection
- Secure server configuration

Implementing strong web server security measures is essential for protecting sensitive user information and maintaining secure online services.

## INTERNSHIP TASK – DAY 7

### DATABASE SECURITY ANALYSIS AND PROTECTION OF VISA PROCESSING WEBSITE

#### Aim

To improve the security of the database used by the Visa Processing/Consultancy website by analyzing access control, securing authentication, preventing SQL Injection, encrypting sensitive data, enabling monitoring and backups, and documenting security findings and recommendations.

#### Objectives

The objectives of this practical are:

1. To analyze database user access control and permissions.
2. To secure database authentication mechanisms.
3. To prevent SQL Injection vulnerabilities using secure coding practices.
4. To study encryption techniques for protecting sensitive data.
5. To implement database logging and backup strategies.
6. To document database security findings, fixes, and recommendations.

#### Tools Used

| Tool                         | Purpose                                    |
|------------------------------|--------------------------------------------|
| Burp Suite Community Edition | Request analysis and SQL Injection testing |
| MySQL / PostgreSQL Research  | Database security study                    |
| bcrypt / Argon2              | Password encryption                        |
| SQL Queries                  | Database access analysis                   |
| Visa Consultancy Website     | Security analysis target                   |

#### Website Used for Testing

Website:

<https://veerconsultancyglobal.com>

The website was selected because it represents a Visa Processing/Consultancy platform suitable for database security analysis and cybersecurity research.

## PART 1 — ANALYZING USER ACCESS CONTROL

### Introduction

Database access control determines which users can:

- Read data
- Modify data
- Delete records
- Manage databases

Improper permissions may allow attackers or unauthorized users to access sensitive information.

### Step 1 — Checking Database User Roles and Permissions

The database security structure was conceptually analyzed to understand role-based permissions.

Typical Database Roles:

- Administrator
- Application User
- Read-Only User
- Backup Operator

### Risks of Excessive Privileges

If users have unnecessary privileges, attackers may:

- Delete records
- Modify user data
- Access confidential information
- Execute administrative commands

### Recommended Principle — Least Privilege

#### Description

Users should only receive permissions required for their tasks.

#### Example

| User Type        | Required Permissions       |
|------------------|----------------------------|
| Application User | SELECT, INSERT             |
| Administrator    | Full Administrative Access |
| Reporting User   | SELECT Only                |

### Recommended Access Control Practices

1. Remove unused database accounts.

2. Restrict administrative access.
3. Use Role-Based Access Control (RBAC).
4. Review permissions regularly.

### **Benefits**

- Reduces insider threats.
- Prevents unauthorized modifications.
- Improves database security.

## **PART 2 — SECURING DATABASE AUTHENTICATION**

### **Introduction**

Weak database authentication may expose sensitive data and backend systems to attackers.

### **Step 2 — Verifying Strong Password Usage**

Strong passwords should be used for:

- Database administrator accounts
- Application database users
- Backup accounts

### **Recommended Password Policy**

1. Minimum 12-character passwords.
2. Combination of:
  - Uppercase letters
  - Lowercase letters
  - Numbers
  - Special symbols
3. Avoid dictionary words.
4. Rotate passwords regularly.

### **Example Strong Passwords**

Example 1:

DB@Secure2026

Example 2:

Str0ng#DatabaseAccess

### **Multi-Factor Authentication (MFA)**

## Description

MFA provides an additional authentication layer beyond passwords.

## Benefits

1. Prevents unauthorized access.
2. Reduces credential attack risks.
3. Improves administrator account security.

## MFA Methods

| MFA Method             | Example              |
|------------------------|----------------------|
| OTP Verification       | SMS/Email codes      |
| Authentication Apps    | Google Authenticator |
| Biometric Verification | Fingerprint/Face ID  |

## Recommended Authentication Improvements

1. Enable MFA for administrators.
2. Restrict remote database access.
3. Use secure VPN connections.
4. Disable default accounts.

## PART 3 — PREVENTION OF SQL INJECTION

### Introduction

SQL Injection occurs when attackers manipulate SQL queries using malicious input.

This may allow:

- Authentication bypass
- Database access
- Data modification
- Data deletion

### Step 3 — Identifying Direct SQL Queries

The Visa Consultancy website was analyzed for input handling and SQL query risks.

Example of Insecure Query:

```
SELECT * FROM users WHERE email = '$email'
```

### Security Risk

Attackers may inject malicious SQL commands into input fields.

Example Payload:

' OR '1'='1

#### **Step 4 — Replacing with Prepared Statements**

Prepared statements separate SQL commands from user input.

#### **Secure Query Example**

SELECT \* FROM users WHERE email = ?

#### **Benefits of Parameterized Queries**

1. Prevent SQL Injection.
2. Improve query security.
3. Separate user input from SQL logic.

#### **Additional SQL Injection Prevention Techniques**

1. Validate user input.
2. Sanitize dangerous characters.
3. Use stored procedures carefully.
4. Restrict database permissions.

### **PART 4 — ENCRYPTION OF SENSITIVE DATA**

#### **Introduction**

Sensitive data stored in databases should be encrypted to prevent unauthorized access.

Sensitive Information Includes:

- Passwords
- Passport details
- Personal information
- Financial data

#### **Step 5 — Encrypting Passwords**

Passwords should never be stored in plain text.

#### **bcrypt Password Hashing**

##### **Description**

bcrypt is a secure password hashing algorithm.

##### **Example bcrypt Hash**

\$2b\$12\$8Fh2K1n4fB3DkT9sQxL2OeXvM7rZP4YjA6nKpW1uV5hQ8eC0dL9uS

**Argon2 Password Hashing**

**Description**

Argon2 is a modern password hashing algorithm resistant to brute-force attacks.

**Benefits of Password Hashing**

- 1. Protects credentials during database leaks.
- 2. Prevents plain-text password exposure.
- 3. Improves authentication security.

**Encrypting Personal and Financial Information**

**Recommended Encryption Methods**

| Data Type        | Recommended Protection      |
|------------------|-----------------------------|
| Passwords        | bcrypt / Argon2             |
| Personal Details | AES Encryption              |
| Financial Data   | Strong Symmetric Encryption |
| Session Tokens   | Secure Token Encryption     |

**Benefits of Encryption**

- Protects confidential information.
- Prevents unauthorized access.
- Reduces data breach impact.

**PART 5 — DATABASE LOGGING AND BACKUPS**

**Introduction**

Logging and backup systems help detect attacks and recover from failures or data loss.

**Step 6 — Enabling Database Logging**

Database logging helps track:

- Failed login attempts
- Suspicious queries
- Unauthorized access
- Administrative changes

**Example MySQL Logging Configuration**

SET GLOBAL general\_log = 'ON';

## Benefits of Logging

1. Detect suspicious activity.
2. Support incident investigation.
3. Monitor database access.

## Log Analysis Tools

| Tool      | Purpose                 |
|-----------|-------------------------|
| Splunk    | Security monitoring     |
| ELK Stack | Centralized logging     |
| Graylog   | Log analysis            |
| Fail2Ban  | Block suspicious access |

## Step 7 — Automated Database Backup

### Importance of Backups

Backups protect against:

- Data loss
- Ransomware attacks
- Hardware failure
- Human error

### Example MySQL Backup Command

```
mysqldump -u root -p database_name > backup.sql
```

### Backup Recommendations

1. Schedule automatic backups.
2. Store backups securely.
3. Encrypt backup files.
4. Test backup restoration regularly.

### Benefits of Automated Backups

- Quick recovery after failures.
- Reduced downtime.
- Improved disaster recovery.

## PART 6 — FINDINGS, FIXES, AND RECOMMENDATIONS

## Security Findings

| Issue Identified             | Observation                                |
|------------------------------|--------------------------------------------|
| Potential SQL Injection Risk | Direct query risks identified conceptually |
| Weak Authentication Risks    | MFA not clearly visible                    |
| Sensitive Data Exposure Risk | Encryption improvements recommended        |
| Logging Improvements Needed  | Advanced monitoring recommended            |
| Backup Mechanism Not Visible | Automated backup recommended               |

## Fixes Implemented or Recommended

| Security Issue            | Recommended Fix                    |
|---------------------------|------------------------------------|
| SQL Injection Risk        | Use prepared statements            |
| Weak Password Protection  | Use bcrypt/Argon2                  |
| Excessive User Privileges | Apply least privilege principle    |
| Missing MFA               | Enable Multi-Factor Authentication |
| Weak Monitoring           | Enable centralized logging         |
| Backup Risks              | Configure automated backups        |

## Additional Security Recommendations

1. Enable database encryption for sensitive data.
2. Restrict remote database access.
3. Use secure VPN connections for administrators.
4. Monitor suspicious database activity continuously.
5. Rotate administrator passwords regularly.
6. Remove unused database accounts.
7. Enable secure backup encryption.
8. Perform regular security audits.

## Practical Observations During Testing

During testing and research activities:

1. HTTPS communication was observed.
2. POST requests transmitted data securely.

3. Potential SQL Injection risks were studied conceptually.
4. Password hashing mechanisms such as bcrypt and Argon2 were researched.
5. Database logging and backup techniques were analyzed.

These observations highlighted the importance of strong database security protections for protecting sensitive user information.

### **Findings and Analysis**

The practical demonstrated the importance of database security in protecting Visa Processing and Consultancy systems. SQL Injection prevention, secure authentication, encryption, logging, and backups are essential for maintaining database confidentiality, integrity, and availability.

Prepared statements and parameterized queries significantly reduce SQL Injection risks, while password hashing algorithms such as bcrypt and Argon2 improve credential security.

Continuous monitoring, least privilege access control, and automated backups are necessary for maintaining secure database operations.

### **Conclusion**

The practical successfully demonstrated database security analysis and protection techniques for a Visa Processing/Consultancy website.

The activity improved understanding of:

- Database access control
- SQL Injection prevention
- Secure authentication
- Password hashing
- Data encryption
- Database monitoring
- Backup and recovery mechanisms

Strong database security implementation is essential for protecting sensitive user information and maintaining secure online services in modern web applications.

## INTERNSHIP TASK – DAY 8

### API SECURITY ANALYSIS AND PROTECTION OF VISA PROCESSING WEBSITE

#### Aim

To strengthen the security of APIs used in the Visa Processing/Consultancy website by analyzing authentication mechanisms, validating API inputs, implementing rate limiting, securing data transmission, enabling API monitoring, and documenting security findings and recommendations.

#### Objectives

The objectives of this practical are:

1. To analyze API authentication mechanisms.
2. To implement secure API authentication using JWT.
3. To validate API inputs and prevent injection attacks.
4. To understand and implement rate limiting.
5. To secure API communication using HTTPS encryption.
6. To enable API logging and monitoring.
7. To document API security findings, fixes, and recommendations.

#### Tools Used

| Tool                         | Purpose                  |
|------------------------------|--------------------------|
| Burp Suite Community Edition | API request analysis     |
| Browser Developer Tools      | Inspect API requests     |
| JWT (JSON Web Token)         | API authentication       |
| HTTPS/TLS Protocol           | Secure communication     |
| Visa Consultancy Website     | Security analysis target |

#### Website Used for Testing

Website:

<https://veerconsultancyglobal.com>

The website was selected because it represents a Visa Processing/Consultancy platform suitable for API security analysis and cybersecurity research.

## PART 1 — ANALYZING API AUTHENTICATION

### Introduction

APIs are used to exchange data between frontend applications and backend servers. Weak API authentication may allow unauthorized access to sensitive user information.

### Step 1 — Checking API Authentication Mechanism

The Visa Consultancy website was analyzed using Burp Suite and browser developer tools to inspect API communication.

Steps Performed:

1. Opened website in Burp Browser.
2. Opened:  
Proxy → HTTP History
3. Submitted forms and monitored API requests.
4. Inspected request headers and responses.

### Observations

| Security Feature      | Observation         |
|-----------------------|---------------------|
| HTTPS Enabled         | Yes                 |
| JSON Communication    | Observed            |
| Authentication Tokens | Not Clearly Visible |
| Session Handling      | Present             |

### Risks of Weak API Authentication

Weak authentication may lead to:

- Unauthorized access
- Account compromise
- API abuse
- Data leakage

### JWT (JSON Web Token) Authentication

#### Description

JWT is a secure token-based authentication mechanism used in APIs.

A JWT contains:

1. Header
2. Payload
3. Signature

### **Example JWT Structure**

header.payload.signature

### **Benefits of JWT Authentication**

1. Secure user authentication.
2. Stateless authentication mechanism.
3. Reduced server-side session storage.
4. Improved API security.

### **Example Authorization Header**

Authorization: Bearer <JWT\_TOKEN>

### **Recommended Authentication Improvements**

1. Implement JWT authentication.
2. Use token expiration.
3. Secure token storage.
4. Enable Multi-Factor Authentication (MFA).

## **PART 2 — VALIDATING API INPUTS**

### **Introduction**

Improper API input validation may allow:

- SQL Injection
- XSS attacks
- Malicious payload submission
- API abuse

### **Step 2 — Identifying API Endpoints Accepting User Input**

The following input areas were analyzed:

- Contact forms
- Inquiry forms
- Email fields
- Search parameters

### **Example API Request Payload**

```
{  
  "name": "User",
```

```
"email": "user@example.com",  
"message": "Visa Inquiry"  
}
```

### Risks of Improper Validation

Attackers may submit:

- Malicious SQL payloads
- JavaScript code
- Oversized requests
- Invalid data formats

### SQL Injection Payload Example

```
' OR '1'='1
```

### XSS Payload Example

```
<script>alert('XSS')</script>
```

### Recommended Input Validation Techniques

| Validation Technique   | Purpose                    |
|------------------------|----------------------------|
| Length Validation      | Prevent oversized payloads |
| Email Validation       | Accept valid email formats |
| Numeric Validation     | Restrict numeric fields    |
| Character Filtering    | Prevent malicious payloads |
| Server-Side Validation | Prevent bypass attacks     |

### Secure API Validation Recommendations

1. Validate all API parameters.
2. Sanitize dangerous characters.
3. Reject invalid data formats.
4. Limit request size.
5. Use parameterized queries.

### Benefits of Input Validation

- Prevents injection attacks.
- Improves API reliability.

- Protects backend systems.

## **PART 3 — IMPLEMENTATION OF RATE LIMITING**

### **Introduction**

Rate limiting restricts the number of API requests allowed within a specific time period.

It helps prevent:

- API abuse
- Brute-force attacks
- Denial-of-Service (DoS) attacks
- Excessive traffic

### **Step 3 — Researching Rate Limiting**

Rate limiting is commonly implemented using:

- IP-based restrictions
- Token-based limits
- Request counters
- Time windows

### **Example Rate Limit Policy**

| <b>Limit Type</b>     | <b>Example</b> |
|-----------------------|----------------|
| Requests Per Minute   | 100            |
| Login Attempts        | 5              |
| API Requests Per Hour | 1000           |

### **Example Nginx Rate Limiting Configuration**

```
limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;
```

### **Example Express.js Rate Limiting**

```
const rateLimit = require("express-rate-limit");
```

```
const limiter = rateLimit({  
  windowMs: 15 * 60 * 1000,  
  max: 100  
});
```

### **Benefits of Rate Limiting**

1. Prevents API abuse.
2. Protects against brute-force attacks.
3. Reduces server overload.
4. Improves application availability.

### **Recommended Rate Limiting Practices**

1. Limit authentication attempts.
2. Apply limits to sensitive endpoints.
3. Monitor excessive traffic.
4. Block suspicious IP addresses.

## **PART 4 — ENCRYPTING DATA IN TRANSIT**

### **Introduction**

Sensitive API communication should always be encrypted to prevent interception.

### **Step 4 — Checking HTTPS Communication**

The Visa Consultancy website was inspected for HTTPS usage.

Observation:

HTTPS communication was enabled.

### **Security Headers Observed**

| Security Header           | Observation |
|---------------------------|-------------|
| Strict-Transport-Security | Present     |
| HTTPS                     | Enabled     |

### **Importance of HTTPS**

HTTPS protects against:

- Credential theft
- Session hijacking
- Data interception
- Man-in-the-Middle attacks

### **Encrypting Sensitive API Responses**

Sensitive data such as:

- Personal details
- Passport information

- Financial data

should be encrypted during transmission.

### Recommended Encryption Methods

| Data Type         | Recommended Protection |
|-------------------|------------------------|
| API Communication | HTTPS/TLS              |
| Passwords         | bcrypt / Argon2        |
| Sensitive Data    | AES Encryption         |
| Tokens            | Secure JWT Signing     |

### Benefits of Encryption

1. Protects confidential data.
2. Prevents unauthorized interception.
3. Improves communication security.

## PART 5 — API LOGGING AND MONITORING

### Introduction

API monitoring helps detect suspicious activity and security threats.

### Step 5 — Enabling API Logging

API logging helps track:

- Failed requests
- Unauthorized access attempts
- Excessive traffic
- Suspicious payloads

### Example API Log Entry

POST /api/login - 401 Unauthorized

### Recommended API Logging Information

1. Request method
2. API endpoint
3. Timestamp
4. IP address
5. Response status code

## API Monitoring Tools

| Tool      | Purpose                 |
|-----------|-------------------------|
| Splunk    | Log analysis            |
| ELK Stack | Centralized monitoring  |
| Graylog   | Security event analysis |
| Fail2Ban  | Block suspicious IPs    |

## Suspicious API Requests Identified

Conceptual suspicious behaviors studied:

1. Repeated failed login requests.
2. Excessive API requests.
3. Injection payload attempts.
4. Unauthorized access attempts.

## Recommended Monitoring Improvements

1. Enable centralized logging.
2. Monitor failed authentication attempts.
3. Detect abnormal request patterns.
4. Configure automated alerts.

## Benefits of API Monitoring

- Early attack detection.
- Faster incident response.
- Improved security visibility.

## PART 6 — FINDINGS, FIXES, AND RECOMMENDATIONS

### Security Findings

| Issue Identified   | Observation                       |
|--------------------|-----------------------------------|
| HTTPS Enabled      | Yes                               |
| JWT Authentication | Not Clearly Visible               |
| Input Validation   | Additional validation recommended |
| Rate Limiting      | Not Clearly Visible               |
| API Monitoring     | Advanced monitoring recommended   |
| Injection Risks    | Input sanitization required       |

### Fixes Implemented or Recommended

| Security Issue           | Recommended Fix              |
|--------------------------|------------------------------|
| Weak Authentication      | Implement JWT authentication |
| Injection Risks          | Validate and sanitize inputs |
| Excessive API Requests   | Configure rate limiting      |
| Weak Monitoring          | Enable API logging           |
| Sensitive Data Exposure  | Encrypt sensitive responses  |
| Unauthorized Access Risk | Implement MFA                |

### API Security Best Practices

1. Use HTTPS for all API communication.
2. Implement JWT or OAuth authentication.
3. Validate all user inputs.
4. Apply rate limiting on APIs.
5. Enable centralized API logging.
6. Encrypt sensitive data.
7. Restrict unauthorized access.
8. Monitor suspicious API traffic continuously.

### Practical Observations During Testing

During practical testing and research:

1. HTTPS communication was enabled.
2. JSON-based API communication was observed.
3. HSTS security header was present.
4. Authentication token mechanisms were not clearly visible.
5. Input validation and rate limiting protections were not clearly observable.

These observations highlighted the importance of implementing layered API security protections.

### **Findings and Analysis**

The practical demonstrated the importance of API security in protecting Visa Processing and Consultancy systems. Secure authentication, input validation, encryption, rate limiting, and monitoring are essential for maintaining API confidentiality, integrity, and availability.

JWT authentication improves API access security, while input validation and rate limiting reduce injection and abuse risks. Continuous monitoring and HTTPS encryption further strengthen API protection.

### **Conclusion**

The practical successfully demonstrated API security analysis and protection techniques for a Visa Processing/Consultancy website.

The activity improved understanding of:

- API authentication
- JWT security
- Input validation
- Rate limiting
- HTTPS encryption
- API monitoring
- Secure API practices

Strong API security implementation is essential for protecting sensitive user information and maintaining secure online services in modern web applications.

## INTERNSHIP TASK – DAY 9

### BASIC PENETRATION TESTING OF VISA PROCESSING WEBSITE

#### Aim

To perform basic penetration testing on a Visa Processing/Consultancy website by identifying vulnerabilities related to authentication, SQL Injection, XSS, session management, and file uploads using security testing tools.

#### Objectives

The objectives of this practical are:

1. To set up a penetration testing environment.
2. To perform automated vulnerability scanning.
3. To test for SQL Injection vulnerabilities.
4. To test for Cross-Site Scripting (XSS) vulnerabilities.
5. To assess authentication and session management security.
6. To analyze file upload security.
7. To document vulnerabilities, risk levels, and recommended fixes.

#### Tools Used

| Tool                           | Purpose                          |
|--------------------------------|----------------------------------|
| Burp Suite Community Edition   | Request interception and testing |
| OWASP ZAP                      | Automated vulnerability scanning |
| Mozilla Firefox / Burp Browser | Website interaction              |
| SQL Injection Payloads         | Database security testing        |
| XSS Payloads                   | Script injection testing         |
| Visa Consultancy Website       | Security analysis target         |

#### Website Used for Testing

Website:

<https://veerconsultancyglobal.com>

The website was selected because it represents a Visa Processing/Consultancy platform suitable for penetration testing and cybersecurity analysis.

## PART 1 — SETTING UP THE TESTING ENVIRONMENT

### Introduction

A controlled testing environment is important for safely performing penetration testing activities without affecting production systems.

### **Step 1 — Installing Security Testing Tools**

The following tools were installed and configured:

1. Burp Suite Community Edition
2. OWASP ZAP
3. Mozilla Firefox / Burp Browser

### **Purpose of Tools**

| <b>Tool</b>             | <b>Function</b>                  |
|-------------------------|----------------------------------|
| Burp Suite              | Intercept and modify requests    |
| OWASP ZAP               | Automated vulnerability scanning |
| Browser Developer Tools | Inspect requests and headers     |

### **Step 2 — Configuring Burp Suite**

Steps Performed:

1. Opened Burp Suite.
2. Opened:  
Proxy → Options
3. Verified proxy listener:

127.0.0.1:8080

4. Opened Burp Browser.
5. Accessed the website.

Observation:

Traffic interception was successfully enabled.

## **PART 2 — AUTOMATED VULNERABILITY SCANNING**

### **Introduction**

Automated vulnerability scanners help identify common web application vulnerabilities quickly.

### **Step 3 — Performing Vulnerability Scan**

OWASP ZAP and Burp Suite were used to scan the website.

### **Scan Targets**

The following areas were analyzed:

- Web pages
- Forms
- HTTP headers
- API requests
- Authentication mechanisms

### **Vulnerabilities Checked**

| <b>Vulnerability Type</b>   | <b>Purpose</b>            |
|-----------------------------|---------------------------|
| Outdated Libraries          | Detect insecure software  |
| Weak Authentication         | Identify login weaknesses |
| Missing Security Headers    | Browser security analysis |
| SSL/TLS Issues              | Communication security    |
| Input Validation Weaknesses | Injection attack analysis |

### **Observations**

1. HTTPS communication was enabled.
2. HSTS security header was present.
3. Some advanced security headers were not clearly visible.
4. Backend timeout responses were observed occasionally.

### **Importance of Automated Scanning**

Automated scanning helps:

- Detect common vulnerabilities
- Improve security assessment speed
- Identify configuration weaknesses

## **PART 3 — SQL INJECTION TESTING**

### **Introduction**

SQL Injection occurs when attackers manipulate SQL queries using malicious user input.

### **Step 4 — Identifying Input Fields**

The following input fields were analyzed:

- Contact forms
- Inquiry forms

- Search parameters
- Email fields

### SQL Injection Payloads Used

' OR '1'='1

admin' --

### Testing Procedure

1. Entered SQL payloads into input fields.
2. Submitted requests through Burp Suite.
3. Observed responses for:
  - SQL errors
  - Authentication bypass
  - Unusual responses

### Observations

No direct SQL Injection vulnerability was confirmed during testing.

However, secure server-side validation and parameterized queries are strongly recommended.

### Risks of SQL Injection

1. Unauthorized database access.
2. Data leakage.
3. Authentication bypass.
4. Data deletion or modification.

### Recommended Fixes

| Security Improvement   | Purpose                       |
|------------------------|-------------------------------|
| Prepared Statements    | Prevent SQL Injection         |
| Input Validation       | Restrict malicious input      |
| Parameterized Queries  | Separate SQL logic from input |
| Least Privilege Access | Restrict database permissions |

### Secure Query Example

SELECT \* FROM users WHERE email = ?

## PART 4 — XSS (CROSS-SITE SCRIPTING) TESTING

## Introduction

XSS vulnerabilities occur when malicious JavaScript executes inside a user's browser.

### Step 5 — Testing for XSS Vulnerabilities

The following payloads were tested:

```
<script>alert('XSS')</script>
```

```
<img src=x onerror=alert(1)>
```

### Testing Procedure

1. Inserted payloads into form fields.
2. Submitted requests.
3. Observed browser behavior for script execution.

### Observations

Injected scripts were not visibly executed during testing.

However, additional sanitization and output encoding are recommended.

### Risks of XSS

1. Session hijacking.
2. Cookie theft.
3. Credential theft.
4. Malicious redirection.

### Recommended Fixes

| Security Improvement    | Purpose                  |
|-------------------------|--------------------------|
| Input Sanitization      | Remove dangerous code    |
| Output Encoding         | Prevent script execution |
| Content-Security-Policy | Reduce XSS risks         |
| Character Filtering     | Restrict dangerous input |

## PART 5 — AUTHENTICATION AND SESSION MANAGEMENT TESTING

### Introduction

Weak authentication and session management may allow attackers to gain unauthorized access.

### Step 6 — Authentication Analysis

The following areas were analyzed:

- Login mechanisms
- Session cookies
- Password handling
- Password reset behavior

### Session Handling Observations

| Security Feature | Observation         |
|------------------|---------------------|
| HTTPS Enabled    | Yes                 |
| Session Cookies  | Present             |
| MFA              | Not Clearly Visible |
| CAPTCHA          | Not Clearly Visible |
| Secure Headers   | Present             |

### Session Hijacking Analysis

Session handling was analyzed conceptually using intercepted requests.

Potential Risks:

- Session hijacking
- Session fixation
- Weak session expiration

### Weak Password Attack Analysis

Weak password risks were studied conceptually.

Risks Include:

- Brute-force attacks
- Credential stuffing
- Unauthorized account access

### Recommended Authentication Improvements

1. Enable Multi-Factor Authentication (MFA).
2. Use strong password policies.
3. Implement account lockout mechanisms.
4. Use secure session cookies.
5. Regenerate session IDs after login.

## PART 6 — FILE UPLOAD SECURITY TESTING

### Introduction

Improper file upload validation may allow attackers to upload malicious files.

### Step 7 — File Upload Analysis

The website was checked for:

- File upload forms
- Document submission functionality
- Attachment upload options

### Risks of Insecure File Uploads

1. Malware upload.
2. Remote code execution.
3. Unauthorized file access.
4. Server compromise.

### Security Checks Performed

| Security Check       | Purpose                  |
|----------------------|--------------------------|
| File Type Validation | Restrict dangerous files |
| File Size Limits     | Prevent abuse            |
| File Renaming        | Prevent path attacks     |
| Malware Scanning     | Detect malicious uploads |

### Recommended File Upload Security Measures

1. Allow only approved file types.
2. Rename uploaded files securely.
3. Store uploads outside public directories.
4. Scan files for malware.
5. Restrict executable file uploads.

### Example Restricted File Types

Allowed:

.pdf

.jpg

.png

Blocked:

.php

.exe

.js

## PART 7 — FINDINGS, RISK LEVELS, AND RECOMMENDATIONS

### Security Findings

| Vulnerability                     | Observation                         | Risk Level |
|-----------------------------------|-------------------------------------|------------|
| SQL Injection Risk                | No direct exploitation confirmed    | Medium     |
| XSS Risk                          | Additional sanitization recommended | Medium     |
| Weak Authentication Protections   | MFA not visible                     | Medium     |
| Missing Advanced Security Headers | CSP/X-Frame-Options unclear         | Medium     |
| Potential File Upload Risks       | Validation protections recommended  | Medium     |
| HTTPS Communication               | Enabled                             | Low Risk   |

### Recommended Fixes

| Security Issue      | Recommended Fix                      |
|---------------------|--------------------------------------|
| SQL Injection Risk  | Use prepared statements              |
| XSS Risk            | Implement sanitization and CSP       |
| Weak Authentication | Enable MFA and CAPTCHA               |
| Session Security    | Use secure cookies and expiration    |
| File Upload Risks   | Restrict file types and scan uploads |
| Weak Monitoring     | Enable logging and alerts            |

### Additional Security Recommendations

1. Implement Content-Security-Policy headers.
2. Use bcrypt or Argon2 for password hashing.
3. Apply rate limiting on authentication endpoints.
4. Monitor suspicious traffic continuously.
5. Perform regular penetration testing.

6. Keep server software updated.
7. Enable centralized logging and monitoring.

### **Practical Observations During Testing**

During practical testing:

1. HTTPS communication was enabled.
2. HSTS security header was present.
3. Input validation protections were partially visible.
4. No direct SQL Injection or XSS exploitation was confirmed.
5. Advanced authentication protections such as MFA were not clearly visible.

These observations highlighted the importance of layered security protections in web applications.

### **Findings and Analysis**

The practical demonstrated the importance of penetration testing in identifying weaknesses within Visa Processing and Consultancy systems. SQL Injection testing, XSS analysis, authentication testing, and file upload security checks are essential for maintaining secure web applications.

Burp Suite and OWASP ZAP helped analyze requests, identify potential vulnerabilities, and understand security protections implemented by the website.

Strong authentication, secure input validation, HTTPS encryption, secure file handling, and continuous monitoring are critical for reducing cybersecurity risks.

### **Conclusion**

The practical successfully demonstrated basic penetration testing techniques for a Visa Processing/Consultancy website.

The activity improved understanding of:

- Vulnerability scanning
- SQL Injection testing
- XSS testing
- Authentication security
- Session management
- File upload security
- Security assessment methodologies

Implementing strong cybersecurity protections and conducting regular penetration testing are essential for protecting sensitive user information and maintaining secure online services.

## INTERNSHIP TASK – DAY 10

### INCIDENT RESPONSE SIMULATION FOR VISA PROCESSING WEBSITE

#### Aim

To simulate an incident response scenario for a Visa Processing/Consultancy website by identifying a cyber attack, containing the threat, investigating suspicious activities, eradicating vulnerabilities, recovering systems, and documenting the incident response process.

#### Objectives

The objectives of this practical are:

1. To understand common cyber security incidents affecting web applications.
2. To learn incident containment and isolation procedures.
3. To analyze logs and investigate suspicious activities.
4. To study threat eradication and vulnerability remediation techniques.
5. To understand recovery and backup restoration procedures.
6. To document the incident response lifecycle and lessons learned.

#### Tools Used

| Tool                         | Purpose                             |
|------------------------------|-------------------------------------|
| Burp Suite Community Edition | Traffic and request analysis        |
| Browser Developer Tools      | Network inspection                  |
| Log Monitoring Tools         | Incident investigation              |
| Backup and Recovery Concepts | System restoration                  |
| Visa Consultancy Website     | Security incident simulation target |

#### Website Used for Simulation

Website:

<https://veerconsultancyglobal.com>

The website was selected because it represents a Visa Processing/Consultancy platform suitable for cybersecurity incident response simulation and security analysis.

## PART 1 — IDENTIFYING A SECURITY INCIDENT

### Introduction

Web applications are vulnerable to many cyber attacks that may compromise:

- User accounts

- Sensitive information
- Server infrastructure
- Database systems

Common cyber threats include:

- Malware attacks
- Phishing attacks
- SQL Injection
- Cross-Site Scripting (XSS)
- Credential theft
- Data breaches

### **Selected Incident Scenario — SQL Injection Attack**

#### **Description**

A simulated SQL Injection attack scenario was selected for this practical.

In this scenario:

An attacker attempts to exploit vulnerable input fields on the Visa Processing website to gain unauthorized access to the database.

#### **Example Malicious Payload**

' OR '1'='1

#### **Potential Impact on Visa Processing Website**

| Impact Area           | Possible Consequences               |
|-----------------------|-------------------------------------|
| User Accounts         | Unauthorized access                 |
| Personal Information  | Data leakage                        |
| Database              | Modification or deletion of records |
| Authentication System | Login bypass                        |
| Business Operations   | Service disruption                  |

#### **Risks Associated with the Incident**

1. Exposure of applicant information.
2. Unauthorized database access.
3. Data tampering or deletion.
4. Reputation damage.

5. Legal and compliance risks.

## **Importance of Incident Response**

A rapid incident response helps:

- Minimize damage
- Prevent attack spread
- Restore operations quickly
- Protect sensitive information

## **PART 2 — CONTAINING THE THREAT**

### **Introduction**

Containment is the process of isolating affected systems to prevent the attack from spreading further.

### **Immediate Containment Actions**

#### **Step 1 — Isolate Affected Systems**

The affected web server or application module should be temporarily isolated from external access.

#### **Actions Performed**

1. Restricted suspicious IP addresses.
2. Disabled vulnerable application endpoints.
3. Limited database access temporarily.
4. Monitored suspicious traffic.

#### **Step 2 — Block Malicious Requests**

Firewall and Web Application Firewall (WAF) rules should be updated to block:

- Malicious payloads
- SQL Injection attempts
- Suspicious IP addresses

#### **Example Firewall Action**

Block suspicious IP address access

#### **Step 3 — Disable Compromised Accounts**

If unauthorized access is suspected:

1. Reset passwords.
2. Revoke active sessions.

3. Enable Multi-Factor Authentication (MFA).

### Preventing Attack Spread

| Prevention Method  | Purpose                    |
|--------------------|----------------------------|
| Network Isolation  | Stop attack propagation    |
| Firewall Rules     | Block malicious traffic    |
| Session Revocation | Remove unauthorized access |
| WAF Protection     | Filter malicious requests  |

### Importance of Threat Containment

Containment helps:

- Reduce attack impact
- Protect unaffected systems
- Prevent data loss

## PART 3 — INVESTIGATING THE INCIDENT

### Introduction

Incident investigation helps determine:

- How the attack occurred
- Which systems were affected
- What data was exposed

### Step 4 — Log Analysis

System logs and web server logs were conceptually analyzed to identify suspicious activity.

### Logs Reviewed

| Log Type            | Purpose                   |
|---------------------|---------------------------|
| Access Logs         | Track requests            |
| Error Logs          | Detect application errors |
| Authentication Logs | Detect failed logins      |
| Database Logs       | Detect suspicious queries |

### Example Suspicious Log Entry

POST /login ' OR '1'=1

## **Indicators of Suspicious Activity**

1. Repeated failed login attempts.
2. Unusual SQL query patterns.
3. Unexpected traffic spikes.
4. Unauthorized API requests.

## **Identifying the Entry Point**

The suspected entry point was:

- User input fields lacking proper validation.

## **Nature of the Attack**

Attack Type:

- SQL Injection

Attack Goal:

- Unauthorized database access

## **Importance of Incident Investigation**

Investigation helps:

- Understand attacker behavior
- Identify vulnerabilities
- Improve future defenses

## **PART 4 — ERADICATING THE THREAT**

### **Introduction**

Eradication removes malicious activity and fixes vulnerabilities exploited during the attack.

### **Step 5 — Removing the Vulnerability**

The vulnerable input handling mechanism was conceptually fixed using:

- Input validation
- Parameterized queries
- Prepared statements

### **Example Secure SQL Query**

```
SELECT * FROM users WHERE email = ?
```

### **Step 6 — Applying Security Patches**

The following improvements were recommended:

1. Update web server software.
2. Patch outdated libraries.
3. Enable security headers.
4. Improve authentication controls.

### Malware Removal Research

If malware is detected:

1. Scan infected systems.
2. Remove malicious files.
3. Restore clean application versions.
4. Perform integrity verification.

### Recommended Security Improvements

| Security Improvement | Purpose                   |
|----------------------|---------------------------|
| Prepared Statements  | Prevent SQL Injection     |
| Security Updates     | Patch vulnerabilities     |
| Input Validation     | Restrict malicious input  |
| MFA                  | Strengthen authentication |
| WAF                  | Block malicious traffic   |

### Importance of Threat Eradication

Eradication helps:

- Remove malicious activity
- Prevent reinfection
- Improve application security

## PART 5 — RECOVERING THE SYSTEM

### Introduction

Recovery restores affected systems safely after threat removal.

### Step 7 — Restoring Systems Using Backups

Backups are essential for restoring:

- Databases
- Website files

- Configurations

### **Backup Recovery Process**

1. Verify clean backup integrity.
2. Restore affected systems.
3. Reconfigure security settings.
4. Test application functionality.

### **Security Controls Before Recovery**

Before bringing systems online:

1. Verify vulnerabilities are fixed.
2. Reset compromised passwords.
3. Enable logging and monitoring.
4. Confirm firewall protections.

### **Recovery Recommendations**

| <b>Recovery Action</b>    | <b>Purpose</b>           |
|---------------------------|--------------------------|
| Restore Clean Backups     | Recover lost data        |
| Re-enable Services Safely | Prevent reinfection      |
| Verify Security Controls  | Improve protection       |
| Monitor Traffic           | Detect recurring attacks |

### **Importance of Recovery**

Recovery ensures:

- Business continuity
- Secure restoration of services
- Reduced downtime

## **PART 6 — INCIDENT REPORT AND LESSONS LEARNED**

### **Incident Summary**

A simulated SQL Injection attack scenario was analyzed for the Visa Processing website.

The attack targeted vulnerable user input fields and attempted unauthorized database access using malicious SQL payloads.

### **Incident Timeline**

| Phase         | Activity                          |
|---------------|-----------------------------------|
| Detection     | Suspicious SQL payload identified |
| Containment   | Restricted malicious traffic      |
| Investigation | Analyzed logs and requests        |
| Eradication   | Improved input validation         |
| Recovery      | Planned secure restoration        |

### Vulnerabilities Identified

| Vulnerability         | Risk Level |
|-----------------------|------------|
| Weak Input Validation | High       |
| SQL Injection Risk    | High       |
| Missing MFA           | Medium     |
| Weak Monitoring       | Medium     |

### Security Fixes Applied or Recommended

| Security Fix           | Purpose                  |
|------------------------|--------------------------|
| Prepared Statements    | Prevent SQL Injection    |
| Input Sanitization     | Restrict malicious input |
| Logging and Monitoring | Detect attacks           |
| MFA                    | Improve authentication   |
| Security Updates       | Patch vulnerabilities    |

### Lessons Learned

1. Input validation is critical for preventing SQL Injection.
2. Continuous monitoring improves attack detection.
3. Strong authentication reduces unauthorized access risks.
4. Regular backups are essential for recovery.
5. Incident response planning improves security readiness.

### Additional Security Recommendations

1. Conduct regular penetration testing.

2. Use Web Application Firewalls (WAF).
3. Implement centralized logging.
4. Enable Multi-Factor Authentication.
5. Apply regular software updates.
6. Monitor suspicious traffic continuously.
7. Perform security awareness training.

### **Practical Observations During Simulation**

During the simulated incident response activity:

1. SQL Injection risks were analyzed conceptually.
2. HTTPS communication was observed.
3. HSTS security header was present.
4. Input validation improvements were recommended.
5. Logging and monitoring mechanisms were studied.

These observations highlighted the importance of rapid incident response and layered security protections.

### **Findings and Analysis**

The practical demonstrated the importance of incident response procedures in protecting Visa Processing and Consultancy systems. Detection, containment, investigation, eradication, and recovery are essential stages for minimizing the impact of cyber attacks.

Prepared statements, logging, secure authentication, backups, and continuous monitoring significantly improve cybersecurity resilience.

### **Conclusion**

The practical successfully demonstrated an incident response simulation for a Visa Processing/Consultancy website.

The activity improved understanding of:

- Cyber attack identification
- Threat containment
- Log analysis
- Vulnerability remediation
- Backup recovery
- Security monitoring
- Incident response documentation

Strong incident response planning and cybersecurity controls are essential for protecting sensitive user information and maintaining secure online services.

## INTERNSHIP TASK – DAY 11

### IMPLEMENTATION OF SECURE CODING PRACTICES FOR VISA PROCESSING WEBSITE

#### Aim

To implement secure coding practices for the Visa Processing/Consultancy website by reviewing insecure code, validating user input, securing authentication systems, encrypting sensitive data, removing insecure dependencies, and documenting security improvements.

#### Objectives

The objectives of this practical are:

1. To identify security issues in existing application code.
2. To implement secure input validation techniques.
3. To strengthen authentication and authorization mechanisms.
4. To encrypt sensitive user information.
5. To remove insecure and outdated dependencies.
6. To document secure coding improvements and code changes.

#### Tools Used

| Tool                             | Purpose                           |
|----------------------------------|-----------------------------------|
| Burp Suite Community Edition     | Security testing                  |
| Browser Developer Tools          | Request inspection                |
| bcrypt / Argon2                  | Password hashing                  |
| npm audit / pip-audit            | Dependency vulnerability scanning |
| Express Validator / Django Forms | Input validation                  |
| Visa Consultancy Website         | Security analysis target          |

#### Website Used for Testing

Website:

<https://veerconsultancyglobal.com>

The website was selected because it represents a Visa Processing/Consultancy platform suitable for secure coding analysis and cybersecurity research.

## PART 1 — REVIEWING EXISTING CODE FOR SECURITY ISSUES

### Introduction

Insecure coding practices may expose applications to:

- SQL Injection
- Cross-Site Scripting (XSS)
- Credential theft
- Unauthorized access
- Data breaches

The application code and functionality were conceptually reviewed to identify security weaknesses.

## **Step 1 — Checking for Hardcoded Credentials and API Keys**

### **Description**

Hardcoded credentials inside source code may expose:

- Database passwords
- API tokens
- Administrator credentials

### **Example of Insecure Code**

```
const dbPassword = "admin123";  
const apiKey = "ABC123SECRET";
```

### **Security Risks**

1. Credential exposure.
2. Unauthorized access.
3. API abuse.
4. Data compromise.

### **Secure Approach**

Sensitive credentials should be stored using:

- Environment variables
- Secret management systems

### **Secure Example**

```
const dbPassword = process.env.DB_PASSWORD;  
const apiKey = process.env.API_KEY;
```

## **Step 2 — Identifying Weak Input Validation**

The following user input areas were analyzed:

- Login forms

- Contact forms
- Search fields
- Registration forms

### **Risks of Improper Validation**

Improper validation may allow:

- SQL Injection
- XSS attacks
- Malicious payload execution

### **Observations**

Additional server-side validation and sanitization were recommended for improving security.

## **PART 2 — IMPLEMENTING SECURE INPUT VALIDATION**

### **Introduction**

Input validation ensures that only safe and properly formatted data is accepted by the application.

### **Step 3 — Using Validation Libraries**

#### **Node.js Example — express-validator**

```
const { body, validationResult } = require('express-validator');
```

```
app.post('/login',  
[  
  body('email').isEmail(),  
  body('password').isLength({ min: 8 })  
],  
(req, res) => {  
  const errors = validationResult(req);  
  
  if (!errors.isEmpty()) {  
    return res.status(400).json({ errors: errors.array() });  
  }  
});
```

#### **Python Example — Django Forms**

```
from django import forms
```

```
class LoginForm(forms.Form):  
    email = forms.EmailField()  
    password = forms.CharField(min_length=8)
```

**Benefits of Validation Libraries**

- 1. Validate input formats.
- 2. Restrict malicious input.
- 3. Improve application reliability.
- 4. Prevent injection attacks.

**Step 4 — Sanitizing Special Characters**

**Example XSS Payload**

```
<script>alert('XSS')</script>
```

**Risks of Unsanitized Input**

- 1. XSS attacks.
- 2. Session hijacking.
- 3. Credential theft.

**Recommended Sanitization Techniques**

| Technique          | Purpose                        |
|--------------------|--------------------------------|
| Character Escaping | Prevent script execution       |
| Input Sanitization | Remove malicious code          |
| Output Encoding    | Prevent browser interpretation |
| Length Validation  | Restrict oversized input       |

**Importance of Input Validation**

Secure input validation protects against:

- SQL Injection
- Cross-Site Scripting
- Malicious payload execution

**PART 3 — SECURING AUTHENTICATION AND AUTHORIZATION**

## Introduction

Weak authentication systems increase the risk of unauthorized access and account compromise.

## Step 5 — Implementing Role-Based Access Control (RBAC)

### Description

RBAC restricts access based on user roles.

### Example Roles

| Role  | Permissions               |
|-------|---------------------------|
| Admin | Full system access        |
| Staff | Limited management access |
| User  | Application access only   |

### Benefits of RBAC

1. Restricts unauthorized actions.
2. Reduces privilege abuse.
3. Improves access management.

## Step 6 — Implementing Secure Password Hashing

Passwords should never be stored in plain text.

### Example of Insecure Password Storage

```
password = "user123";
```

### Secure bcrypt Example

```
const bcrypt = require('bcrypt');
```

```
const hashedPassword = await bcrypt.hash(password, 12);
```

### Example bcrypt Hash

```
$2b$12$8Fh2K1n4fB3DkT9sQxL2OeXvM7rZP4YjA6nKpW1uV5hQ8eC0dL9uS
```

## Argon2 Password Hashing

### Description

Argon2 is a modern password hashing algorithm resistant to brute-force attacks.

### Benefits of Secure Password Hashing

1. Protects credentials during database leaks.

2. Prevents plain-text password exposure.
3. Improves authentication security.

### **Recommended Authentication Improvements**

1. Enable Multi-Factor Authentication (MFA).
2. Use strong password policies.
3. Implement session expiration.
4. Regenerate session IDs after login.

## **PART 4 — ENCRYPTING SENSITIVE DATA**

### **Introduction**

Sensitive user information should be encrypted both:

- At rest
- During transmission

### **Step 7 — Encrypting Sensitive Information**

Sensitive data includes:

- Passwords
- Financial information
- Personal details
- Passport information

### **Recommended Encryption Methods**

| <b>Data Type</b> | <b>Recommended Protection</b> |
|------------------|-------------------------------|
| Passwords        | bcrypt / Argon2               |
| Sensitive Data   | AES Encryption                |
| Communication    | HTTPS/TLS                     |
| API Tokens       | Secure Token Signing          |

### **HTTPS Communication Analysis**

Observation:

HTTPS communication was enabled on the Visa Consultancy website.

### **Benefits of HTTPS**

1. Prevents credential theft.
2. Protects transmitted data.

3. Prevents Man-in-the-Middle attacks.

### **Example Secure HTTPS URL**

`https://veerconsultancyglobal.com`

## **PART 5 — REMOVING UNUSED AND INSECURE DEPENDENCIES**

### **Introduction**

Outdated or vulnerable dependencies may expose applications to known security vulnerabilities.

### **Step 8 — Running Dependency Security Checks**

#### **Node.js — npm audit**

`npm audit`

#### **Python — pip-audit**

`pip-audit`

### **Purpose of Dependency Checks**

Dependency scanning helps identify:

- Vulnerable packages
- Outdated libraries
- Known security flaws

### **Updating Vulnerable Dependencies**

#### **Node.js Example**

`npm update`

#### **Python Example**

`pip install --upgrade package_name`

### **Risks of Outdated Dependencies**

1. Remote code execution.
2. Authentication bypass.
3. Data leakage.
4. Server compromise.

### **Recommended Dependency Security Practices**

1. Remove unused packages.
2. Update libraries regularly.
3. Monitor vulnerability reports.

4. Use trusted packages only.

## PART 6 — DOCUMENTING SECURITY IMPROVEMENTS

### Security Improvements Implemented

| Security Improvement  | Purpose                   |
|-----------------------|---------------------------|
| Environment Variables | Protect credentials       |
| Input Validation      | Prevent injection attacks |
| Output Sanitization   | Prevent XSS               |
| bcrypt/Argon2 Hashing | Secure passwords          |
| RBAC                  | Restrict access           |
| HTTPS                 | Secure communication      |
| Dependency Updates    | Patch vulnerabilities     |

### Before and After Secure Coding Examples

#### Example 1 — Hardcoded Credential Removal

##### Before

```
const password = "admin123";
```

##### After

```
const password = process.env.DB_PASSWORD;
```

#### Example 2 — SQL Injection Prevention

##### Before

```
SELECT * FROM users WHERE email = '$email'
```

##### After

```
SELECT * FROM users WHERE email = ?
```

#### Example 3 — Password Security

##### Before

```
password = "user123";
```

##### After

```
const hashedPassword = await bcrypt.hash(password, 12);
```

### Security Findings

| Security Issue        | Observation                           |
|-----------------------|---------------------------------------|
| Hardcoded Credentials | Security risk identified conceptually |
| Weak Input Validation | Additional validation recommended     |
| SQL Injection Risks   | Parameterized queries required        |
| Weak Password Storage | bcrypt/Argon2 recommended             |
| Outdated Dependencies | Regular updates recommended           |

### Additional Security Recommendations

1. Conduct regular code reviews.
2. Enable centralized logging.
3. Implement Multi-Factor Authentication.
4. Use secure session management.
5. Perform dependency audits regularly.
6. Apply Content-Security-Policy headers.
7. Use HTTPS for all communication.

### Practical Observations During Testing

During practical testing and research:

1. HTTPS communication was enabled.
2. HSTS security header was observed.
3. Secure password hashing techniques were researched.
4. Dependency auditing tools were studied.
5. Input validation and sanitization improvements were analyzed.

These observations highlighted the importance of secure coding practices for protecting sensitive user information and reducing cyber security risks.

### Findings and Analysis

The practical demonstrated the importance of secure coding practices in protecting Visa Processing and Consultancy systems. Input validation, secure authentication, password hashing, encryption, dependency management, and RBAC are essential for maintaining application security.

Using bcrypt/Argon2, parameterized queries, HTTPS encryption, and secure dependency management significantly improves application security and reduces attack risks.

## **Conclusion**

The practical successfully demonstrated implementation of secure coding practices for a Visa Processing/Consultancy website.

The activity improved understanding of:

- Secure coding principles
- Input validation
- Authentication security
- RBAC implementation
- Password hashing
- Dependency auditing
- Secure data encryption

Strong secure coding practices are essential for protecting sensitive user information and maintaining secure online services in modern web applications.

## INTERNSHIP TASK – DAY 12

### INSTALLATION AND EXPLORATION OF BURP SUITE FOR VISA PROCESSING WEBSITE

#### Aim

To install and explore Burp Suite for analyzing the security of a Visa Processing/Consultancy website by intercepting HTTP requests, identifying sensitive data transmission, performing basic vulnerability scanning, and documenting security findings.

#### Objectives

The objectives of this practical are:

1. To install and configure Burp Suite.
2. To set up proxy interception for web traffic analysis.
3. To intercept and analyze HTTP/HTTPS requests.
4. To perform a basic vulnerability scan.
5. To identify security weaknesses in the web application.
6. To document findings and recommended fixes.

#### Tools Used

| Tool                         | Purpose                 |
|------------------------------|-------------------------|
| Burp Suite Community Edition | Web security testing    |
| Burp Browser                 | Request interception    |
| Mozilla Firefox              | Proxy configuration     |
| Browser Developer Tools      | Request inspection      |
| Visa Consultancy Website     | Security testing target |

#### Website Used for Testing

Website:

<https://veerconsultancyglobal.com>

The website was selected because it represents a Visa Processing/Consultancy platform suitable for security testing and HTTP request analysis.

## PART 1 — INSTALLATION AND LAUNCH OF BURP SUITE

### Introduction

Burp Suite is a web security testing tool used for:

- Intercepting HTTP/HTTPS traffic

- Analyzing requests and responses
- Identifying vulnerabilities
- Performing penetration testing

### **Step 1 — Downloading Burp Suite**

Burp Suite Community Edition was downloaded from the official website.

### **Step 2 — Installing Burp Suite**

Installation Steps:

1. Opened the installer.
2. Selected installation directory.
3. Completed setup process.
4. Launched Burp Suite.

### **Step 3 — Starting a Temporary Project**

Steps Performed:

1. Opened Burp Suite.
2. Selected:  
Temporary Project
3. Clicked:  
Next
4. Selected:  
Use Burp Defaults
5. Clicked:  
Start Burp

### **Step 4 — Verifying Proxy Listener**

Burp Suite proxy settings were checked.

Steps:

1. Opened:  
Proxy → Options
2. Verified proxy listener:

127.0.0.1:8080

Observation:

Proxy listener was active successfully.

### **Step 5 — Opening Burp Browser**

Steps:

1. Opened:  
Proxy → Intercept
2. Clicked:  
Open Browser

Purpose:

To browse websites through Burp Suite proxy interception.

### **Importance of Burp Suite**

Burp Suite helps:

- Analyze web traffic
- Identify vulnerabilities
- Test application security
- Perform penetration testing

## **PART 2 — INTERCEPTING AND ANALYZING REQUESTS**

### **Introduction**

Burp Suite can intercept HTTP and HTTPS requests between the browser and the server.

### **Step 6 — Enabling Request Interception**

Steps Performed:

1. Opened:  
Proxy → Intercept
2. Enabled:

Intercept is ON

### **Step 7 — Opening the Website**

Website Accessed:

<https://veerconsultancyglobal.com>

Observation:

HTTP/HTTPS requests started appearing in Burp Suite.

### **Step 8 — Capturing Requests**

The following request information was analyzed:

- Request method
- URL
- Headers

- Cookies
- Form data

### Example HTTP Request

POST /contact HTTP/1.1

Host: veerconsultancyglobal.com

Content-Type: application/x-www-form-urlencoded

### Sensitive Data Analysis

The following sensitive information was checked:

- Login credentials
- Session cookies
- Email addresses
- Contact information

### Observations

| Security Feature | Observation |
|------------------|-------------|
| HTTPS Enabled    | Yes         |
| Session Cookies  | Present     |
| POST Requests    | Used        |
| HSTS Header      | Present     |

### Risks of Sensitive Data Exposure

If HTTPS is not used:

- Credentials may be intercepted.
- Session hijacking may occur.
- Sensitive information may leak.

### Importance of Request Analysis

Request analysis helps:

- Detect insecure communication
- Identify weak authentication
- Analyze API behavior

## PART 3 — PERFORMING A BASIC SECURITY SCAN

**Introduction**

Burp Suite helps identify common web application vulnerabilities.

**Step 9 — Using HTTP History**

Steps Performed:

- 1. Opened:  
Proxy → HTTP History
- 2. Reviewed captured requests and responses.
- 3. Analyzed headers and parameters.

**Vulnerabilities Checked**

| Vulnerability Type       | Purpose                       |
|--------------------------|-------------------------------|
| Missing Security Headers | Browser protection analysis   |
| Weak Authentication      | Login security analysis       |
| Input Validation Issues  | Injection attack detection    |
| HTTPS Misconfiguration   | Secure communication analysis |

**Step 10 — Testing Input Fields**

The following input fields were tested:

- Contact forms
- Email fields
- Search fields

**SQL Injection Payloads Tested**

' OR '1'='1

**XSS Payloads Tested**

<script>alert('XSS')</script>

**Observations**

- 1. HTTPS communication was enabled.
- 2. HSTS security header was present.
- 3. No direct SQL Injection exploitation was confirmed.
- 4. Additional input validation and sanitization were recommended.

**Common Security Weaknesses Identified**

| Weakness                              | Risk Level |
|---------------------------------------|------------|
| Missing Advanced Security Headers     | Medium     |
| Potential Input Validation Weaknesses | Medium     |
| MFA Not Clearly Visible               | Medium     |
| File Upload Protections Not Verified  | Medium     |

### Recommended Security Improvements

1. Implement Content-Security-Policy headers.
2. Use parameterized SQL queries.
3. Enable Multi-Factor Authentication.
4. Apply strict input validation.
5. Use secure session handling.

## PART 4 — SECURITY REPORT AND DOCUMENTATION

### Introduction

Security reports help document vulnerabilities, risks, and recommended fixes.

### Step 11 — Capturing Screenshots

Screenshots captured included:

1. Burp Suite dashboard
2. Intercepted HTTP requests
3. HTTP History tab
4. Request headers
5. Security testing activities

### Suggested Screenshot Areas

| Screenshot          | Purpose                   |
|---------------------|---------------------------|
| Proxy Intercept Tab | Show intercepted requests |
| HTTP History        | Show captured traffic     |
| Request Headers     | Analyze security headers  |
| Input Testing       | Document security testing |

## Security Findings

| Vulnerability                        | Observation                    | Risk Level |
|--------------------------------------|--------------------------------|------------|
| HTTPS Enabled                        | Communication secured          | Low        |
| HSTS Header Present                  | Browser HTTPS enforcement      | Low        |
| Input Validation Improvements Needed | Potential injection risks      | Medium     |
| Missing Advanced Security Headers    | Additional protection required | Medium     |
| MFA Not Clearly Visible              | Authentication risk            | Medium     |

## Suggested Fixes

| Security Issue        | Recommended Fix                   |
|-----------------------|-----------------------------------|
| Weak Input Validation | Validate and sanitize inputs      |
| Missing CSP Header    | Implement Content-Security-Policy |
| Weak Authentication   | Enable MFA                        |
| SQL Injection Risks   | Use prepared statements           |
| Session Security      | Use secure cookies                |

## Additional Security Recommendations

1. Conduct regular penetration testing.
2. Enable centralized logging and monitoring.
3. Keep server software updated.
4. Apply rate limiting on login forms.
5. Use secure password hashing algorithms.

## Practical Observations During Testing

During practical testing using Burp Suite:

1. HTTPS communication was active.
2. POST requests transmitted form data securely.
3. Session cookies were observed.
4. HSTS security header was present.
5. No direct SQL Injection exploitation was confirmed.

These observations highlighted the importance of layered security protections and continuous security testing.

### **Findings and Analysis**

The practical demonstrated how Burp Suite can be used for web application security testing and HTTP traffic analysis. Intercepting requests helped analyze headers, cookies, authentication behavior, and data transmission security.

Basic vulnerability analysis demonstrated the importance of:

- HTTPS communication
- Input validation
- Security headers
- Secure authentication
- Session protection

Burp Suite proved useful for identifying potential security weaknesses and improving cybersecurity awareness.

### **Conclusion**

The practical successfully demonstrated installation and exploration of Burp Suite for security testing of a Visa Processing/Consultancy website.

The activity improved understanding of:

- Burp Suite installation
- Proxy configuration
- Request interception
- HTTP/HTTPS analysis
- Vulnerability testing
- Security reporting

Regular security testing using tools such as Burp Suite is essential for protecting sensitive user information and maintaining secure web applications.

## INTERNSHIP TASK – DAY 14

### SECURITY MONITORING USING SPLUNK FOR VISA PROCESSING WEBSITE

#### Aim

To install and configure Splunk for security monitoring, ingest security logs from different sources, detect suspicious activities using SPL queries, create a security dashboard, and document security findings for the Visa Processing/Consultancy website.

#### Objectives

The objectives of this practical are:

1. To install and configure Splunk Enterprise.
2. To collect and index security logs.
3. To search for suspicious activities using SPL queries.
4. To create a real-time security monitoring dashboard.
5. To analyze security threats and mitigation techniques.
6. To document security findings and recommendations.

#### Tools Used

| Tool                      | Purpose                              |
|---------------------------|--------------------------------------|
| Splunk Enterprise         | Security monitoring and log analysis |
| Windows/Linux System Logs | Event monitoring                     |
| Firewall Logs             | Network security monitoring          |
| Browser Developer Tools   | Traffic inspection                   |
| Visa Consultancy Website  | Security monitoring target           |

#### Website Used for Monitoring

Website:

<https://veerconsultancyglobal.com>

The website was selected because it represents a Visa Processing/Consultancy platform suitable for cybersecurity monitoring and log analysis.

## PART 1 — INSTALLATION AND SETUP OF SPLUNK

### Introduction

Splunk is a Security Information and Event Management (SIEM) tool used for:

- Log collection
- Security monitoring

- Threat detection
- Incident investigation
- Real-time analytics

### **Step 1 — Downloading Splunk Enterprise**

Splunk Enterprise was downloaded from the official Splunk website.

### **Step 2 — Installing Splunk**

Installation Steps:

1. Opened Splunk installer.
2. Accepted license agreement.
3. Selected installation directory.
4. Created administrator username and password.
5. Completed installation process.

### **Step 3 — Launching Splunk**

After installation:

1. Opened Splunk Enterprise.
2. Accessed the web interface using:

`http://localhost:8000`

3. Logged in using administrator credentials.

### **Importance of Splunk**

Splunk helps:

- Monitor security events
- Detect cyber attacks
- Analyze logs
- Generate alerts
- Investigate incidents

## **PART 2 — INGESTING SECURITY LOGS**

### **Introduction**

Logs contain important information about:

- User activity
- Authentication attempts

- Firewall events
- System errors
- Security incidents

#### Step 4 — Adding Data Sources

The following log sources were configured:

| Data Source         | Purpose                           |
|---------------------|-----------------------------------|
| System Logs         | Monitor operating system activity |
| Firewall Logs       | Detect network attacks            |
| Application Logs    | Monitor web application behavior  |
| Authentication Logs | Detect login failures             |

#### Steps to Add Logs in Splunk

1. Opened:  
Settings → Add Data
2. Selected log source type.
3. Uploaded or monitored log files.
4. Indexed logs into Splunk.

#### Example Log Sources

| Log File     | Purpose            |
|--------------|--------------------|
| access.log   | Web requests       |
| error.log    | Application errors |
| firewall.log | Network activity   |
| auth.log     | Login activity     |

#### Benefits of Log Collection

1. Detect suspicious behavior.
2. Monitor user activity.
3. Investigate incidents.
4. Improve threat visibility.

### PART 3 — SEARCHING FOR SECURITY THREATS

#### Introduction

Splunk Processing Language (SPL) is used to search and analyze security logs.

**Step 5 — Searching for Failed Login Attempts**

**Example SPL Query**

```
index=main "failed login"
```

**Purpose**

This query searches logs for failed authentication attempts.

**Step 6 — Detecting Brute-Force Attacks**

**Example SPL Query**

```
index=main "failed login"
| stats count by src_ip
| where count > 5
```

**Purpose**

This query identifies IP addresses with repeated failed login attempts.

**Indicators of Brute-Force Attacks**

- 1. Multiple failed logins from same IP.
- 2. Rapid authentication requests.
- 3. Unusual login times.
- 4. Repeated password failures.

**Example Security Threats Analyzed**

| Threat Type         | Description                |
|---------------------|----------------------------|
| Brute-Force Attack  | Repeated password guessing |
| Unauthorized Access | Invalid login attempts     |
| Suspicious Traffic  | Unusual request activity   |
| Web Attacks         | Malicious payload attempts |

**Importance of Threat Detection**

Threat detection helps:

- Identify attacks early
- Prevent unauthorized access
- Improve incident response

# PART 4 — CREATING A SECURITY DASHBOARD

## Introduction

Splunk dashboards provide visual monitoring of security events in real time.

## Step 7 — Creating a Dashboard

Steps Performed:

- 1. Opened:  
Dashboards → Create New Dashboard
- 2. Entered dashboard name.
- 3. Added visualizations using SPL queries.

## Dashboard Components

| Dashboard Widget      | Purpose                     |
|-----------------------|-----------------------------|
| Failed Login Attempts | Detect brute-force attacks  |
| Firewall Events       | Monitor blocked traffic     |
| HTTP Requests         | Monitor application traffic |
| Error Logs            | Detect application issues   |

## Example Dashboard SPL Query

```
index=main
| stats count by sourcetype
```

## Visualization Types Used

- 1. Bar Charts
- 2. Pie Charts
- 3. Event Tables
- 4. Real-Time Event Panels

## Benefits of Security Dashboards

- 1. Real-time monitoring.
- 2. Faster threat detection.
- 3. Improved visibility.
- 4. Easier incident investigation.

# PART 5 — SECURITY ANALYSIS AND REPORTING

## Introduction

Security reports help summarize:

- Threats detected
- Vulnerabilities observed
- Mitigation techniques
- Recommendations

### Security Findings

| Security Issue              | Observation                                            | Risk Level |
|-----------------------------|--------------------------------------------------------|------------|
| Failed Login Attempts       | Repeated authentication failures observed conceptually | Medium     |
| Brute-Force Risk            | Excessive login attempts possible                      | Medium     |
| Monitoring Enabled          | Logs collected successfully                            | Low        |
| Suspicious Traffic Analysis | Conceptual attack monitoring performed                 | Medium     |

### Mitigation Steps Recommended

| Security Threat     | Recommended Mitigation             |
|---------------------|------------------------------------|
| Brute-Force Attacks | Implement account lockout          |
| Unauthorized Access | Enable Multi-Factor Authentication |
| Suspicious Traffic  | Configure firewall rules           |
| Excessive Requests  | Apply rate limiting                |
| Weak Monitoring     | Enable real-time alerts            |

### Recommended Security Improvements

1. Enable centralized logging.
2. Configure real-time alerts.
3. Implement Multi-Factor Authentication.
4. Apply rate limiting for login endpoints.
5. Monitor suspicious IP addresses continuously.
6. Perform regular security audits.
7. Integrate firewall and application logs into Splunk.

## **Example Alert Query**

```
index=main "failed login"  
| stats count by src_ip  
| where count > 10
```

## **Purpose**

Generate alerts for suspicious login activity.

## **Practical Observations During Monitoring**

During practical monitoring activities:

1. Log ingestion was configured successfully.
2. SPL queries were used to analyze login failures.
3. Brute-force detection techniques were researched.
4. Dashboard visualizations were created conceptually.
5. Threat monitoring workflows were studied.

These observations highlighted the importance of SIEM tools for cybersecurity monitoring and incident detection.

## **Findings and Analysis**

The practical demonstrated the importance of security monitoring in protecting Visa Processing and Consultancy systems. Splunk provides centralized visibility into system logs, authentication events, firewall activity, and security incidents.

SPL queries help detect suspicious behaviors such as brute-force attacks and unauthorized access attempts. Dashboards improve visibility and enable faster incident response.

Real-time monitoring, alerting, and log analysis are essential for maintaining strong cybersecurity defenses.

## **Conclusion**

The practical successfully demonstrated security monitoring using Splunk for a Visa Processing/Consultancy website.

The activity improved understanding of:

- Splunk installation
- Log ingestion
- SPL query analysis
- Threat detection
- Dashboard creation

- Security monitoring
- Incident analysis

Continuous monitoring using SIEM tools such as Splunk is essential for detecting cyber threats, improving incident response, and maintaining secure web applications.